LivingDocs — Design and Implementation of an AI-Assisted Self-Updating Document System for Engineering Teams

Design and Implementation of an AI-Assisted Self-Updating Document System for Engineering Teams	
Group Members	Nguyễn Đức Tài Nhân - 075207000402 Nguyễn Hữu Đăng Khôi - 075207003339 Lê Điền Hải Đăng - 079207049465 Nguyễn Lê Minh - 075207004599 Nguyễn Thị Phương Linh - 056307011966 Lâm Gia Huy - 072207009703
Lecturer	Nguyễn Văn Chiến

– Ho Chi Minh, November 2026

Table of Contents

- Definition and Acronyms
- I. Project Introduction
- II. Project Management Plan
- III. Software Requirement Specification
- IV. Software Design Description
- V. Software Testing Documentation
- VI. Release Package and User Guides
- Appendix A. Jira Traceability

Definition and Acronyms

Acronym / Term	Definition	Description
AI	Artificial Intelligence	Used in the project scope for source-code analysis, documentation

		generation, knowledge retrieval, and drift analysis.
API	Application Programming Interface	An interface through which software components communicate. Specific API contracts are not fully described in the reviewed Jira titles.
CI	Continuous Integration	Automated checks that validate changes to the project.
CD	Continuous Deployment / Delivery	Automated preparation or deployment of software to an environment. The precise deployment policy is not fully specified in Jira.
GitHub OAuth	GitHub authorization flow	Used for GitHub-related authentication or account connection.
RAG	Retrieval-Augmented Generation	Listed in the AI Service workstream as knowledge retrieval for documentation-related functions.
SRS	Software Requirements Specification	Describes what the software is expected to do.
SDD	Software Design Description	Describes the proposed architecture, components, data design, and detailed behavior.

UC	Use Case	A user goal or system interaction.
UAT	User Acceptance Testing	Validation that the system supports expected user workflows. Jira includes a System Integration & Acceptance Testing task.
Documentation drift	Difference between source-code behavior or structure and the documentation describing it.	The Jira backlog includes detection, classification, severity assessment, review, and history functions.
AST	Abstract Syntax Tree	Mentioned by the Code Analysis & AST Processing workstream.

I. Project Introduction

1. Overview

1.1 Project Information

Item	Information supported by Jira
Project name	LivingDocs
Jira project key	LD
Product type	Software documentation generation and management system, based on the project backlog
Main project areas	Authentication and user management; GitHub integration; documentation generation; automatic updates; drift detection; version history; knowledge base;

	review and approval; administration; frontend; AI service; CI/CD and testing
Source of project scope	Jira epics, features, tasks, and subtasks

1.2 Project Team

Full Name	Role	MSSV	Gmail
Nguyễn Đức Tài Nhân	Leader	075207000402	nhanndt750402@ut.edu.vn
Nguyễn Hữu Đăng Khôi	Member	075207003339	khoinhd753339@ut.edu.vn
Lê Điền Hải Đăng	Member	079207049465	danglhh799465@ut.edu.vn
Nguyễn Lê Minh	Member	075200700 4599	minhnl754599@ut.edu.vn
Nguyễn Thị Phương Linh	Member	056307011966	linhntp561966@ut.edu.vn
Lâm Gia Huy	Member	072207009703	huylg729703@ut.edu.vn

1.3 Product Background

Software projects evolve as developers change source code, add features, and fix defects. Documentation can become outdated when it is maintained separately from the code. The LivingDocs Jira backlog addresses this problem through documentation generation, automatic documentation updates, documentation drift detection, version history, and a knowledge base.

The project also includes repository synchronization and commit/pull-request ingestion so that source-code changes can provide context for documentation-related workflows. Review and approval features are intended to provide a controlled process for checking documentation before acceptance.

1.4 Existing Systems (in progress)

2. Business Opportunity

The project backlog focuses on reducing manual effort in creating and maintaining software documentation. Its planned capabilities address several practical needs:

- Generate documentation from source code and project context.
- Identify documentation that may be affected by source changes.
- Detect, classify, and assess documentation drift.
- Maintain documentation versions, change logs, comparisons, and rollback capability.
- Search a knowledge base and navigate relationships between code and documentation.
- Use review, approval, and audit functions to support controlled documentation changes.

3. Software Product Vision

LivingDocs aims to provide a common workflow for generating, reviewing, finding, and maintaining software documentation as a project evolves. Its Jira scope combines repository integration, AI-assisted analysis and generation, documentation lifecycle management, administrative controls, and automated quality checks.

4. Project Scope and Limitations

4.1 Major Features

Feature	Feature area	Scope recorded in Jira	Key Jira references
FE1	Authentication and user management	Registration, login/logout, GitHub OAuth, profile and notification preferences, user and role management	LD-5, LD-15, LD-16, LD-17, LD-19, LD-261

FE2	GitHub integration	Account connection, repository management and synchronization, commit/pull-request ingestion, connection notifications	LD-6, LD-22–LD-26
FE3	Documentation generation	Source-code analysis, generation, templates, preview/editing, evidence and submission	LD-7, LD-79–LD-83
FE4	Automatic documentation update	Source-change detection, affected-documentation detection, update workflow, review and rollback	LD-8, LD-112–LD-115
FE5	Documentation drift	Detection, classification, severity assessment, review/resolution, history	LD-9, LD-140–LD-144
FE6	Versions and change history	Version management, change log, diff/comparison, rollback	LD-10, LD-173–LD-176

FE7	Knowledge base	Search, code/documentation relationships, repository and collection management, template management	LD-11, LD-201–LD-204
FE8	Review and approval	Staff review, manager approval, comments/change requests, approval audit trail	LD-12, LD-229–LD-232
FE9	Administration and governance	User/role management, workspace/repository administration, system configuration, audit/monitoring/notifications	LD-13, LD-257–LD-260
FE10	Frontend	Authentication, repository, documentation, review/approval, administration/dashboard interfaces	LD-286–LD-291
FE11	AI service	Code analysis and AST processing, RAG/knowledge retrieval, AI documentation	LD-322–LD-352

		generation, drift analysis, suggestions/confidence	
FE12	CI/CD and testing	CI/CD pipeline, automated testing, GitHub Actions integration, Docker/deployment, system integration and acceptance testing	LD-358–LD-388

4.2 Limitations and Exclusions (in progress)

II. Project Management Plan

1. Overview

1.1 Scope and Estimation (in progress)

1.2 Project Objectives

1. Support account registration, authentication, and role-aware access.
2. Connect to GitHub and synchronize relevant repository information.
3. Analyze source code and generate documentation.
4. Detect source changes and documentation that may need updating.
5. Detect, classify, review, and track documentation drift.
6. Manage documentation versions, change logs, comparisons, and rollback.
7. Provide knowledge-base search and code/documentation navigation.
8. Support documentation review, approval, comments, and audit trails.
9. Provide administrative and monitoring functions.

10. Establish automated build/test checks and a deployment workflow.

1.3 Project Risks (in progress)

2. Management Approach

2.1 Project Process (in progress)

- The project applies the _____ methodology:
- Iteration model:
- Phases:
  - sprint 1 .....

2.2 Quality Management

Quality-related Jira work includes CI/CD, automated testing, GitHub Actions integration, and system integration/acceptance testing.

The intended quality approach is to:

- Run configured build and test checks.
- Validate integration between system components.
- Record test and pipeline outcomes.
- Investigate failed checks before considering a change ready.
- Verify key user workflows against acceptance criteria.

3. Project Deliverables

Deliverable	Jira traceability	Evidence/status note
Authentication and user management functions	LD-5, LD-15–LD-20, LD-257–LD-261	Jira contains feature and implementation tasks; status varies by issue.
GitHub integration	LD-6, LD-22–LD-26	Repository connection, management, synchronization, ingestion, and notifications are tracked.
Documentation generation	LD-7, LD-79–LD-106	Source analysis and generation are tracked,

		along with templates, preview/editing, and evidence/submission.
Automatic documentation updates	LD-8, LD-112–LD-134	Change detection, affected-document detection, updates, review, and rollback are tracked.
Drift detection and review	LD-9, LD-140–LD-167	Detection, classification, severity, review, and history are tracked.
Version and change history	LD-10, LD-173–LD-195	Version management, change log, diff/comparison, and rollback are tracked.
Knowledge base	LD-11, LD-201–LD-223	Search, code/documentation relationship, collection, and template management are tracked.
Review and approval workflow	LD-12, LD-229–LD-251	Staff review, manager approval, comments, and audit trail are tracked.
Administration and governance	LD-13, LD-257–LD-280	User/role, workspace/repository, configuration, monitoring, audit, and notifications are tracked.
Frontend interface	LD-286–LD-316	Authentication UI is marked Done in Jira; other interface tasks

		have their own statuses.
AI service	LD-322–LD-352	Code analysis, RAG, generation, drift analysis, and suggestion/confidence management are tracked.
CI/CD and testing	LD-358–LD-388	Pipeline, automated tests, GitHub Actions, Docker/deployment, and system integration/acceptance testing are tracked.

4. Responsibility Assignments (in progress)

5. Project Communications

Communication Item	**Who / Target**	**Purpose**	**When, Frequency**	Type, Tool, Method(s)
Weekly Planning Chatting	All team members	Plan tasks for upcoming week, discuss blockers	Every 5 Spring, usually monday, 1 session/2 weeks	Zalo chat
Weekly Progress Review	All team members	Share progress, adjust timeline	Every 5 Spring, 1 session/2 weeks	Zalo chat
Internal Feature Demo	Relevant developers + reviewer	Validate completed features before merging	After each feature completion	Zalo chat

Documentation Review	2 assigned members	Review and update SRS, SDR, User Manual	Every 2 weeks	Zalo chat
Urgent Bug or Blocker Sync	Involved members only	Quickly resolve urgent issues	Ad-hoc	Zalo chat
Final Demo Rehearsal	All members	Full demo run-through, time-check, Q&A preparation	Week before present	In-person

6. Configuration Management

6.1 Document Management

The project has a Confluence documentation area containing the project documentation and SRS. The team should use page history and a clear revision process to track document changes.

6.2 Source Code Management (in progress)

- The source code is hosted on GitHub using a standard branching strategy:
  - main: production-ready code.
  -

6.3 Tools and Infrastructure

Category	Tools / Infrastructure
Source repository and integration	GitHub is in scope. Exact OAuth scopes and webhook/event configuration need confirmation.
Frontend	The frontend is a distinct implementation workstream; exact framework/version is not stated in the reviewed issue titles.
AI service	Includes code analysis, AST processing, RAG, generation, drift analysis, and

	suggestion/confidence management.
CI/CD	Includes pipeline and GitHub Actions integration.
Containerization/deployment	Docker and deployment are tracked; final environment details need confirmation.
Automated and system testing	Automated tests and system integration/acceptance testing are tracked.
Documentation and issue tracking	Used for project documentation and requirement/task traceability.

III. Software Requirement Specification

1. Product Overview



The **LivingDocs** is an AI-assisted self-updating documentation system designed to help software engineering teams generate, review, maintain, and update software documentation based on source code and repository changes.

- **Operational Environment:**
  - **Platform:** Web-based application accessible through standard web browsers.
  - **Connectivity:** The system requires network connectivity to access the application, synchronize source code repositories, and communicate with external services.
  - **Infrastructure:** The application consists of a web frontend, a backend service, a GitHub integration service, a code analysis service, and an AI documentation service.

PostgreSQL is used for persistent data storage, while Docker Compose supports local deployment and service orchestration.

- **Anticipated Users:**
  - **Internal Users:** Developers, Staff Reviewers, Managers / Approvers, and System Administrators.
  - **External Systems:** GitHub for source code repository access and change information; GitHub Actions or other configured CI/CD workflows for automated documentation verification.
- **Constraints & Assumptions:**
  - **Language:** The system supports Vietnamese documentation output. The availability of other interface languages is not assumed.
  - **Security:** Access to repositories and documentation operations must follow authentication and authorization rules. GitHub access credentials and internal service communication must be handled securely.
  - **Integrations:** The system integrates with GitHub to retrieve repository information and source code changes. It uses code analysis and AI-assisted generation services to produce documentation and update suggestions.
  - **Human Review:** AI-generated documentation and update suggestions should provide supporting evidence. Documentation changes must follow the configured review and approval workflow before publication, except for documentation types explicitly configured for automatic updates.
  - **Scope Limitation:** The current project focuses on GitHub-based repository integration, code analysis, documentation generation, documentation drift detection, review workflows, and documentation history. Support for other source control platforms, such as GitLab and Azure DevOps, is considered a possible future extension rather than an assumed current capability.
- **Dependencies:**
  - The system relies on GitHub APIs and valid repository access permissions to retrieve source code, repository metadata, and change information.
  - Documentation generation depends on the availability of the code analysis service and AI documentation service. AI generation may operate in a limited or offline mode depending on the configured generator.
  - The system requires a configured PostgreSQL database for persistent application and integration data.
  - The application depends on correct service configuration, network connectivity, and successful communication between the frontend, backend, GitHub integration, code analysis, and AI services.

2. User Requirements

2.1 System Actors

	Actor	Description
1	**Developer**	The primary user who connects a GitHub account, selects repositories, synchronizes source code, selects documentation templates, generates documentation, and views generated results and analysis evidence.
2	**Staff Reviewer**	Responsible for reviewing AI-generated documentation and update suggestions, editing documentation when necessary, and submitting reviewed content for approval.
3	**Manager / Approver**	Responsible for approving or rejecting documentation changes, authorizing publication, and configuring automatic update policies for supported documentation types.
4	**System Administrator**	Responsible for managing user accounts, roles, permissions, and system-level configurations.
5	**GitHub Platform**	Provides repository information, source code, commit history, pull request information, and code changes required for repository synchronization and documentation analysis.
6	**GitHub Actions / CI Pipeline**	Triggers automated documentation verification workflows and receives

		verification results to support continuous integration and documentation quality control.

2.2 Use Cases

2.2.1 Use Case Diagram



2.2.2 Use Case Descriptions

ID	Use Case	Actors	Use Case Description
UC-01	Register Account	Developer, Staff, Manager, Tech Lead	Allows new users to create an account in the system using email credentials or OAuth registration (Task LD-20).
UC-02	Login	All Actors	Authenticates user

			identity via username/password or GitHub OAuth and establishes a secure session (Task LD-38, LD-292).
UC-03	Logout	All Actors	Terminates the active user session securely and clears authentication tokens (Task LD-38).
UC-04	Connect GitHub Account	Developer, Administrator	Links user's personal GitHub account via OAuth2 flow to enable repository access (Task LD-43, LD-53).
UC-05	Browse Available Repositories	Developer	Fetches and displays the list of GitHub repositories accessible by the connected GitHub account (Task LD-63,LD-298).

UC-06	Add Repository to Workspace	Developer, Tech Lead	Adds selected GitHub repositories into a workspace for AI documentation tracking (Task LD-23).
UC-07	Synchronize Repository	Developer, Automated System	Triggers re-synchronization of repository branches, commits, and file changes with LivingDocs (Task LD-63).
UC-08	Analyze Source Code	Developer, Automated System	Parses source code into Abstract Syntax Trees (AST) to extract entities, methods, and signature changes (Task LD-85, LD-328).
UC-09	Select Documentation Template	Developer, Staff	Selects a standardized documentation template (API Reference, README, ADR) for AI generation

			(Task LD-96).
UC-10	Generate Documentation	Developer, Staff	Invokes the AI Engine service to generate documentation content based on code AST structure and selected template (Task LD-90,LD-340).
UC-11	Track Generation Job	Developer, Automated System	Monitors background AI generation progress and pipeline execution status via messaging queues (Task LD-396).
UC-12	View Generated Documentation	Developer, Staff	Renders and displays newly AI-generated documentation drafts within the web interface.
UC-13	View Analysis Evidence	Developer, Staff	Displays grounding Evidence including

			source file paths, AST parsing details, commit hashes, and code diffs (Task LD-106).
UC-14	Handle Generation Failure	Automated System, Administrator	Catches AI processing or parser errors, logs details into Dead Letter Queue (DLQ), and prompt Retries (Task LD-396).
UC-15	Preview Documentation	Developer, Staff	Renders interactive preview of documentation drafts before formal submission or publishing (Task LD-101, LD-105).
UC-16	Edit Documentation	Developer, Staff	Allows manual editing, refinement, and formatting corrections on AI-generated documentation (Task LD- 101).
UC-17	Submit Documentation	Developer	Submits drafted or auto-updated

	for Review		documentation into the review and approval workflow (Task LD-106).
UC-18	Review Documentation	Staff (Reviewer)	Performs first-level review of documentation against source code, templates, and grounding evidence (Task LD-391).
UC-19	Approve Documentation	Staff, Manager	Grants approval at review stage (Staff) or final stage (Manager) to forward docs toward publishing (Task LD- 391).
UC-20	Reject Documentation	Staff, Manager	Rejects submitted documentation with feedback comments and returns it to author for Revision.
UC-21	Publish Documentation	Manager (Approver)	Rejects submitted documentation with feedback comments and

			returns it to author for Revision.
UC-22	Detect Documentation Drift	Automated System, Tech Lead	Automatically scans Pull Requests/commits to identify outdated code references in documentation (Task LD-304).
UC-23	Analyze Code Changes	Automated System	Analyzes PR code diffs and commit evolution to classify changes (Referential, Signature, Semantics (Task LD-85, LD-328).
UC-24	Identify Affected Documentation	Automated System	Uses RAG retrieval to locate all documentation files impacted by modified code elements (Task LD-304).
UC-25	Generate Update Suggestions	Automated System	Generates precise AI update suggestions grounded on verified source

			code changes (Task LD-390).
UC-26	Configure Automatic Updates	Technical Lead, Manager	Defines auto-update policies per repository/module (route to review vs auto-apply).
UC-27	Apply Documentation Update	Developer, Staff	Accepts and applies AI suggested documentation updates to synchronize docs with code (Task LD-304).
UC-28	View Drift Severity	Developer, Staff, Tech Lead	Displays visual indicators for documentation drift severity levels (Low, Medium, High, Critical).
UC-29	View Documentation History	All Actors	Displays complete version history timeline of a document with acting role attributions.
UC-30	Compare Documentation Versions	Developer, Staff, Manager	Renders side-by-side diff view comparing any two

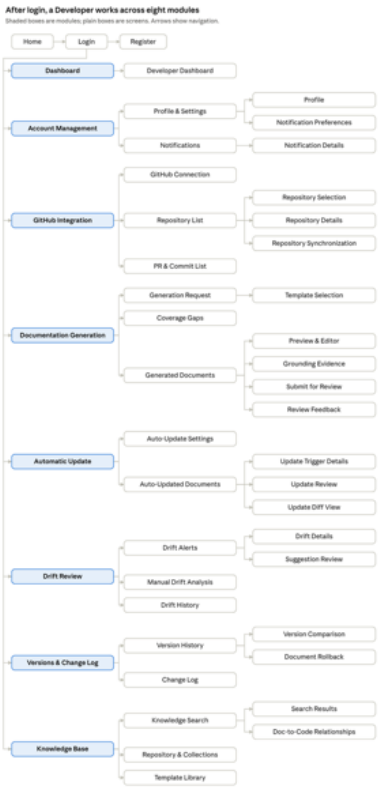
			Document versions.
UC-31	Roll Back Documentation	Manager, Tech Lead, Developer	Restores a document or collection to a previously Approved historical version.
UC-32	Search Documentation	All Actors	Provides full-text and semantic search across knowledge base and linked code entities.
UC-33	Navigate Documentation Knowledge Base	All Actors	Allows browsing repositories, doc collections, and exploring code-to-doc dependency graphs.
UC-34	Manage Users and Roles	Administrator	Manages user accounts, assigns RBAC roles (Developer, Staff, Manager, Tech Lead, Admin) (Task LD-20, LD-38).
UC-35	Manage System Configuration	Administrator	Configures GitHub App

			settings, OAuth secrets, LLM AI parameters, and API integration rules (Task LD-395).
UC-36	Trigger Documentation Verification	Automated System, Developer	Triggers GitHub Actions workflow pipeline to verify documentation drift on Pull Request (Task LD- 376, LD-364, LD-369).
UC-37	Report Verification Results	Automated System, Staff	Posts drift analysis summary, verification status, and PR review comments to GitHub (Task LD-391).

3. Functional Requirements

3.1 System Functional Overview

3.1.1. Web Application Developer Screen Flow



3.1.2 Web Application Developer Screen Description

#	Feature	Screen	Description
1	General	Home	Landing page introducing LivingDocs, with navigation to Login and Register.
2	General	Login	Authenticates the Developer with email and password or GitHub OAuth, then opens the Developer Dashboard.
3	General	Register	Form to create a Developer account with email and password or through GitHub OAuth.
4	Dashboard	Developer Dashboard	Summary of the Developer's repositories, documents by workflow status, pending auto-updates and open drift alerts, with shortcuts to every module.
5	Account Management	Profile & Settings	Entry point to personal profile and notification settings.

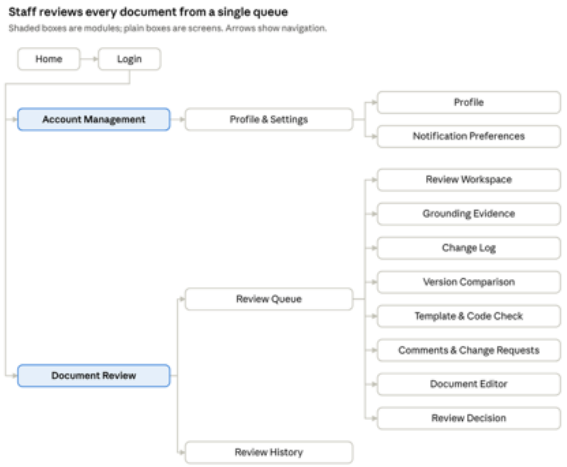
6	Account Management	Profile	View and update personal profile information.
7	Account Management	Notification Preferences	Choose which notifications the Developer receives and how they are delivered.
8	Account Management	Notifications	List of system notifications, such as an invalid GitHub token or webhook, or a failed repository synchronization.
9	Account Management	Notification Details	Full message, time and required action of one notification, for example re-authorizing GitHub.
10	GitHub Integration	GitHub Connection	Authorize or revoke the GitHub account through OAuth, review the repository access scopes requested by the GitHub App, and see the connection status.
11	GitHub Integration	Repository List	Lists repositories of the connected account and those already in the workspace, with connection status, default branch and last synchronization time.
12	GitHub Integration	Repository Selection	Select one or more repositories of the GitHub account and add them to a workspace.
13	GitHub Integration	Repository Details	Shows status, default branch and last synchronization time of one repository, and allows disconnecting it.
14	GitHub Integration	Repository Synchronization	Manually trigger re-synchronization of a repository and follow its progress, result and errors.

15	GitHub Integration	PR & Commit List	Lists the Pull Requests and commits ingested from each connected repository.
16	Documentation Generation	Generation Request	Request automatic documentation for a single file, a selected module or package, or the entire repository.
17	Documentation Generation	Template Selection	Choose the documentation template that controls the structure of the generated document.
18	Documentation Generation	Coverage Gaps	Lists code entities that have no documentation yet and starts generation for them.
19	Documentation Generation	Generated Documents	Lists AI-generated documents with their workflow status: Draft, In Review, Approved, Rejected or Published.
20	Documentation Generation	Preview & Editor	Preview the generated content (classes, methods, endpoints, parameters, return values, dependencies) and edit the draft before submission.
21	Documentation Generation	Grounding Evidence	Shows the source file path, code entity and commit hash behind each generated section.
22	Documentation Generation	Submit for Review	Submits the finished draft into the review and approval workflow.
23	Documentation Generation	Review Feedback	Shows Staff and Manager comments and change requests on a returned document, so the Developer can revise and resubmit.
24	Automatic Update	Auto-Update Settings	Enable or disable automatic documentation update for a repository, module or document.

25	Automatic Update	Auto-Updated Documents	Lists documents that AI updated after a source-code change, with update time and status.
26	Automatic Update	Update Trigger Details	Shows the commit and diff that triggered an automatic update.
27	Automatic Update	Update Review	Accept, edit or dismiss an auto-updated version before it enters approval, or regenerate it on demand.
28	Automatic Update	Update Diff View	Compares the auto-updated version with the currently published version and allows rollback to the version before the update.
29	Drift Review	Drift Alerts	Lists drift alerts generated from Pull Requests, each with a classification and a severity indicator.
30	Drift Review	Drift Details	Shows the code diff, commit and related ticket as evidence, the drift classification (Referential, Signature, Semantic) and the severity (Low, Medium, High, Critical).
31	Drift Review	Suggestion Review	Review the AI-generated update suggestion and its confidence score, then accept, edit or dismiss it.
32	Drift Review	Manual Drift Analysis	Trigger drift analysis for a Pull Request or a specific document.
33	Drift Review	Drift History	History of drift analyses with the resolution status of each issue.
34	Versions & Change Log	Version History	Version timeline of one document.

35	Versions & Change Log	Version Comparison	Shows the difference between any two versions of a document.
36	Versions & Change Log	Document Rollback	Restores a document to a previous version; the restore creates a new version and is written to the change log.
37	Versions & Change Log	Change Log	For each version, shows the acting role (AI, Staff or Manager), the timestamp and the trigger (commit, Pull Request or review action).
38	Knowledge Base	Knowledge Search	Search documentation and linked code entities.
39	Knowledge Base	Search Results	Lists matching documents and code entities, with links to open each one.
40	Knowledge Base	Doc-to-Code Relationships	Shows which code entities a document describes and lets the Developer navigate between a document and its source code.
41	Knowledge Base	Repository & Collections	Browse connected repositories and documentation collections.
42	Knowledge Base	Template Library	Browse the documentation templates available for generation.

3.1.3 Web Application Staff Screen Flow



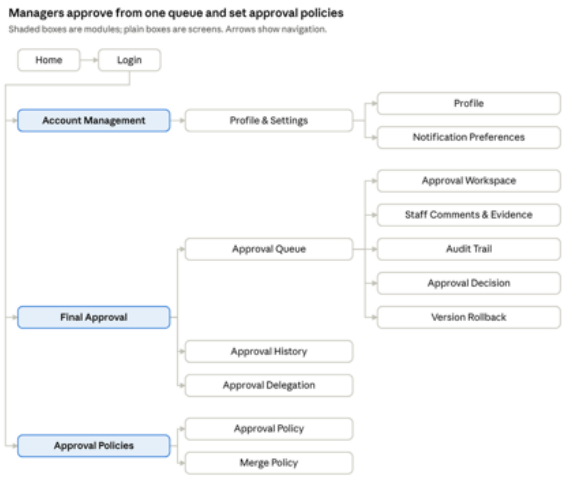
3.1.4 Web Application Staff Screen Description

#	Feature	Screen	Description
1	General	Home	Landing page introducing LivingDocs, with navigation to Login.
2	General	Login	Authenticates the Staff reviewer with email and password or GitHub OAuth, then opens the Review Queue.
3	Account Management	Profile & Settings	Entry point to personal profile and notification settings.
4	Account Management	Profile	View and update personal profile information.
5	Account Management	Notification Preferences	Choose which notifications the reviewer receives and how they are delivered.
6	Document Review	Review Queue	Lists AI-generated and auto-updated documents waiting for first-level review; filter and sort by repository, template, drift severity or submission date.
7	Document Review	Review Workspace	Opens a submitted document for review and is the starting point for evidence,

			comparison and decision.
8	Document Review	Grounding Evidence	Shows the source file, code entity, commit, diff and related ticket of the document under review.
9	Document Review	Change Log	Change history of the document under review, including AI-generated changes and earlier human edits.
10	Document Review	Version Comparison	Compares the submitted version with the last approved version before the decision.
11	Document Review	Template & Code Check	Verify that the document follows the assigned template structure and is consistent with the referenced source code.
12	Document Review	Comments & Change Requests	Add review comments or request changes on specific sections of the document.
13	Document Review	Document Editor	Correct minor issues directly in the document during review.
14	Document Review	Review Decision	Approve the document and forward it to a

			Manager, or reject it and return it to the Developer with feedback.
15	Document Review	Review History	Review history and current status of each document.

3.1.5 Web Application Manager Screen Flow



3.1.6 Web Application Manager Screen Description

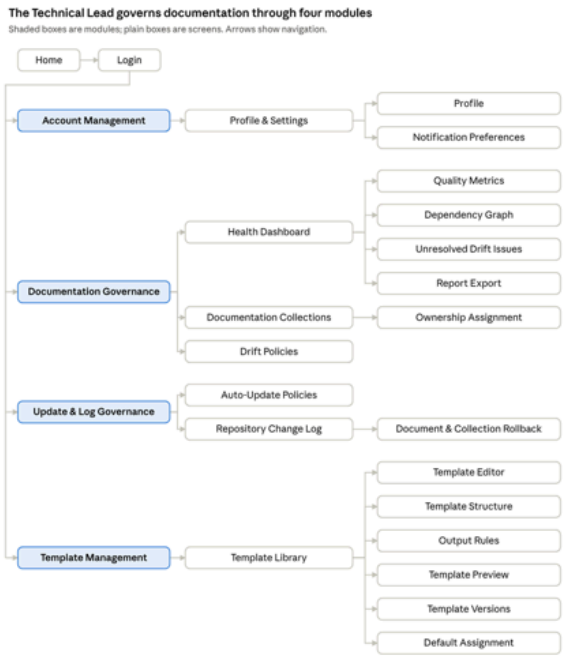
#	Feature	Screen	Description
1	General	Home	Landing page introducing LivingDocs, with navigation to Login.
2	General	Login	Authenticates the Manager with email and password or GitHub OAuth, then opens the Approval Queue.
3	Account Management	Profile & Settings	Entry point to personal profile

			and notification settings.
4	Account Management	Profile	View and update personal profile information.
5	Account Management	Notification Preferences	Choose which notifications the Manager receives and how they are delivered.
6	Final Approval	Approval Queue	Lists documents that passed Staff review and wait for final approval.
7	Final Approval	Approval Workspace	Opens a document for final review and gives access to Staff comments, evidence and the audit trail.
8	Final Approval	Staff Comments & Evidence	Shows the Staff review comments together with the grounding evidence of the document.
9	Final Approval	Audit Trail	Complete audit trail of the document: AI-generated, auto-updated and Staff-edited changes.

10	Final Approval	Approval Decision	Approve the document for publishing or committing to the repository, or reject it or return it to Staff or the Developer for revision; the reason is recorded.
11	Final Approval	Version Rollback	Roll back an approved document to a previous approved version.
12	Final Approval	Approval History	Audit trail of approval decisions: who approved or rejected, when and why.
13	Final Approval	Approval Delegation	Delegate or reassign approval responsibility to another Manager when unavailable.
14	Approval Policies	Approval Policy	Configure which documentation types require Manager approval before publishing.

15	Approval Policies	Merge Policy	Configure whether critical documentation warns about or blocks the merge of a Pull Request.

3.1.7 Web Application Technical Lead Screen Flow



3.1.8 Web Application Technical Lead Screen Description

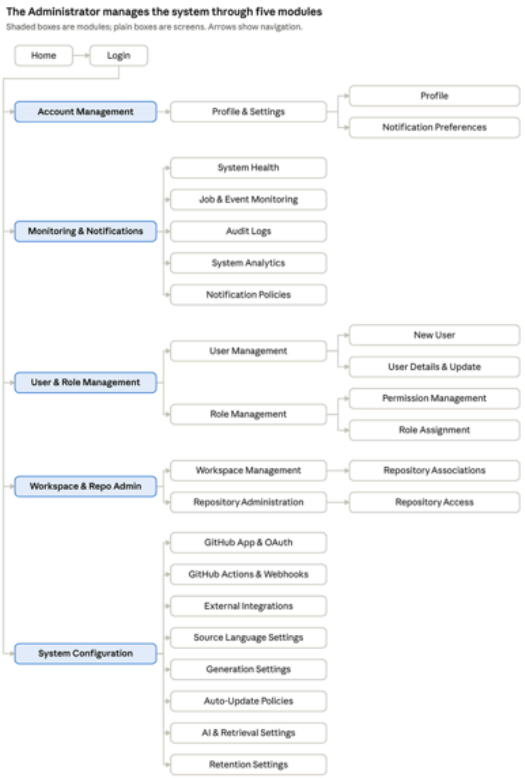
#	Feature	Screen	Description
1	General	Home	Landing page introducing LivingDocs, with navigation to Login.
2	General	Login	Authenticates the Technical Lead with email and password or GitHub OAuth, then opens the Health Dashboard.
3	Account Management	Profile & Settings	Entry point to personal profile and notification

			settings.
4	Account Management	Profile	View and update personal profile information.
5	Account Management	Notification Preferences	Choose which notifications the Technical Lead receives and how they are delivered.
6	Documentation Governance	Health Dashboard	Documentation health overview: freshness, coverage and drift status of repositories and collections.
7	Documentation Governance	Quality Metrics	Documentation quality metrics of each repository.
8	Documentation Governance	Dependency Graph	Documentation-to-code dependency graph used for impact analysis.
9	Documentation Governance	Unresolved Drift Issues	Tracks documentation drift issues that are still open.
10	Documentation Governance	Report Export	Export documentation health and compliance reports.
11	Documentation Governance	Documentation Collections	Create and manage documentation collections.
12	Documentation Governance	Ownership Assignment	Assign an owner to a document or collection.
13	Documentation Governance	Drift Policies	Configure documentation drift detection policies and thresholds.
14	Update & Log Governance	Auto-Update Policies	Configure which repositories, modules or document types update

			automatically when source code changes.
15	Update & Log Governance	Repository Change Log	Tracks version history and change logs across a repository; filter by role (AI, Staff, Manager), document, repository or date range.
16	Update & Log Governance	Document & Collection Rollback	Roll back a document or an entire documentation collection to a previous version.
17	Template Management	Template Library	Browse the default templates (API reference, README, module guide, onboarding guide, ADR); create, clone and delete templates.
18	Template Management	Template Editor	Create or edit a template and its details.
19	Template Management	Template Structure	Define template sections, headings and placeholders mapped to code entities, such as class name, endpoint list or parameter table.
20	Template Management	Output Rules	Configure the formatting and output rules the AI must follow when generating documentation from a template.
21	Template Management	Template Preview	Preview how a template renders before it is published to the template library.

22	Template Management	Template Versions	Maintain template versions and roll back to a previous template version.
23	Template Management	Default Assignment	Assign a default template to each documentation collection or document type.

3.1.9 Web Application Administrator Screen Flow



3.1.10 Web Application Administrator Screen Description

#	Feature	Screen	Description
1	General	Home	Landing page introducing LivingDocs, with navigation to Login.
2	General	Login	Authenticates the Administrator with email and password or GitHub OAuth, then opens System Health.

3	Account Management	Profile & Settings	Entry point to personal profile and notification settings.
4	Account Management	Profile	View and update personal profile information.
5	Account Management	Notification Preferences	Choose which notifications the Administrator receives and how they are delivered.
6	Monitoring & Notifications	System Health	Overall health of the system services and processing pipelines.
7	Monitoring & Notifications	Job & Event Monitoring	Monitor AI processing jobs and event pipelines.
8	Monitoring & Notifications	Audit Logs	View the audit log of administrative operations and system events.
9	Monitoring & Notifications	System Analytics	View system usage and processing analytics.
10	Monitoring & Notifications	Notification Policies	Configure the notification policies of the system.
11	User & Role Management	User Management	Lists system users with their roles and status.
12	User & Role Management	New User	Form to create a user account and assign a role.
13	User & Role Management	User Details & Update	View and update a user, or deactivate the account.

14	User & Role Management	Role Management	Manage the roles Developer, Staff, Manager, Technical Lead and Administrator, including the Staff and Manager roles used in the approval workflow.
15	User & Role Management	Permission Management	Define the permissions granted to each role.
16	User & Role Management	Role Assignment	Assign roles and permissions to users.
17	Workspace & Repo Admin	Workspace Management	Create and manage workspaces.
18	Workspace & Repo Admin	Repository Associations	Manage which repositories belong to each workspace.
19	Workspace & Repo Admin	Repository Administration	Administer the repositories connected to workspaces.
20	Workspace & Repo Admin	Repository Access	Configure repository access and administration settings.
21	System Configuration	GitHub App & OAuth	Configure the GitHub App and the OAuth connection used to link repositories.
22	System Configuration	GitHub Actions & Webhooks	Configure the GitHub Actions and webhook integration.
23	System Configuration	External Integrations	Connect external services: GitHub/GitLab, Jira and Slack.
24	System Configuration	Source Language Settings	Configure the source-code languages supported for AI reading

			and documentation generation.
25	System Configuration	Generation Settings	Configure the default template, output format and generation triggers.
26	System Configuration	Auto-Update Policies	Configure automatic update triggers (on commit, on Pull Request, on merge) and the policy per documentation type: route to review, or auto-apply after approval.
27	System Configuration	AI & Retrieval Settings	Configure the LLM model, embedding model, confidence threshold, token budget, and documentation indexing and retrieval settings.
28	System Configuration	Retention Settings	Configure the retention of the documentation change log and of the audit log.

3.1.11 Screen Authorization

Legend: C = create, R = read, U = update, D = delete, A = approve or reject, E = export, ✓ = every action listed in Typical Actions, - = no access. Delete for accounts means deactivate.

#	Module / Screen	Typical Actions	Developer	Staff	Manager	Technical Lead	Administrator
1	Authentication	Login, logout (Register for Developer)	✓	✓	✓	✓	✓

2	Own Profile & Notification Preferences	R-U	✓	✓	✓	✓	✓
3	Notifications	R	R	-	-	-	-
4	Dashboard / Health views	R	R	-	-	R	R
5	GitHub Connection	C-R-D	C-R-D	-	-	-	-
6	Repository Management & Synchronization	C-R-U-D	C-R-U-D	-	-	-	C-R-U-D
7	Pull Request & Commit data	R	R	R	R	R	-
8	Documentation Generation & Editing	C-R-U	C-R-U	R-U	R	R	-

9	Submit for Review & Workflow Status	C-R	C-R	R	R	R	-
10	Staff Review (queue, comments, decision)	C-R-U-A	R	C-R-U-A	R	R	-
11	Manager Approval (decision, delegation)	R-U-A	R	R	R-U-A	R	-
12	Approval & Merge Policies	C-R-U	-	-	C-R-U	-	-
13	Approval Audit Trail & History	R	-	R	R	R	R
14	Auto-Update Settings & Policies	C-R-U	R-U	-	-	C-R-U	C-R-U
15	Auto-Update	R-U	R-U	R	R	R	-

	Review						
16	Drift Alerts & Review	C-R-U	C-R-U	R	R	R	-
17	Drift Policies & Thresholds	C-R-U	-	-	-	C-R-U	-
18	Version History, Change Log & Comparison	R	R	R	R	R	-
19	Document Rollback	U	U	-	U	U	-
20	Knowledge Base (search, relationships, browse)	R	R	-	-	R	-
21	Documentation Collections & Ownership	C-R-U-D	R	-	-	C-R-U-D	-

22	Documentation Templates	C-R-U-D	R	R	-	C-R-U-D	-
23	Health Metrics, Dependency Graph & Reports	R-E	-	-	-	R-E	-
24	User, Role & Permission Management	C-R-U-D	-	-	-	-	C-R-U-D
25	Workspace & Repository Administration	C-R-U-D	-	-	-	-	C-R-U-D
26	System Configuration	C-R-U	-	-	-	-	C-R-U
27	Monitoring, Audit Logs & Analytics	R	-	-	-	-	R
28	Notification	C-R-U	-	-	-	-	C-R-U

	Policies						

3.1.12 Non-screen Functions

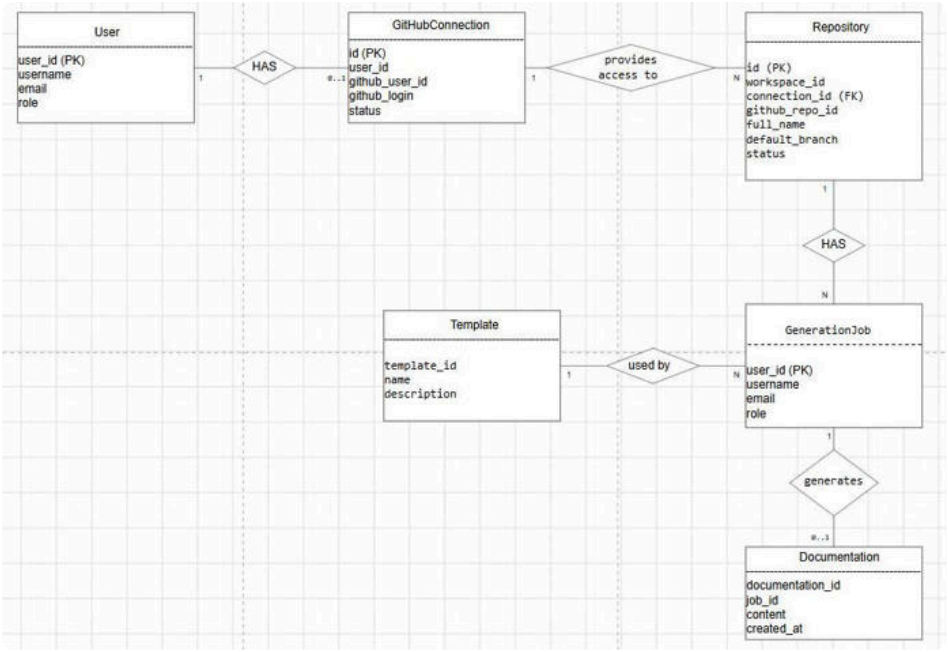
#	Feature	System Function	Description
1	GitHub Integration	Ingest Pull Request and commit events	Receive GitHub Pull Request and commit events, store their data and associate them with the connected repository (LD-68).
2	GitHub Integration	Synchronize repository	Synchronize repository information and the latest changes, track the last synchronization time and handle synchronization failures (LD-63).
3	GitHub Integration	Notify on connection problems	Notify the Developer when GitHub authentication fails, access is revoked, the token or webhook becomes invalid, or a synchronization fails, and record the error (LD-74).
4	Code Analysis	Parse and analyze source code	Read source files of connected repositories, parse them with AST-based analysis, extract code entities (classes, methods, functions, endpoints, parameters, return values) and analyze their dependencies (LD-328). The Code Analyzer receives file content and relative path from the caller (LD-395).
5	Knowledge Retrieval	Index and retrieve (RAG)	Index documentation and code information, generate embeddings, and retrieve

			the documents, code entities and change context used as grounding for AI processing (LD-334).
6	Documentation Generation	Generate documentation	Generate documentation for files, modules, repositories, classes, methods and endpoints from analyzed code and retrieved context, following the selected template, with grounding evidence and a confidence score (LD-340).
7	Orchestration	Repository-to-documentation workflow	Connect repository synchronization, code analysis and AI generation in one backend workflow: validate repository access, pass file content to the Code Analyzer, pass the analysis to generation, and return a clear error for the stage that failed (LD-396).
8	Automatic Update	Detect source changes and update documentation	On a commit or Pull Request, detect changed files and code entities, identify the affected documents with evidence, generate the updated documentation and create a new version; the version goes to Staff review and Manager approval unless a Manager configured auto-apply for that documentation type (LD-116, LD-122, LD-128).
9	Drift Analysis	Detect, classify and rate drift	When a Pull Request is created or updated,

			compare the code change with related documentation; classify drift as Referential, Signature or Semantic; rate severity as Low, Medium, High or Critical; generate evidence; and record the result in the drift history (LD-346, LD-167).
10	AI Suggestion	Suggestions and confidence scoring	Generate documentation-update and drift-resolution suggestions with grounding evidence and a confidence score, apply the configurable confidence threshold, and present every suggestion for human review before it is applied (LD-352).
11	Version & Change Log	Record every documentation change	Create a version for each AI-generated, auto-updated or human-edited change and record the acting role (AI, Staff or Manager), the timestamp and the triggering commit or Pull Request; history is preserved after a rollback (LD-177, LD-195).
12	Approval Control	Enforce the approval gate	Publish or commit documentation only after final Manager approval, apply the merge policy (warn or block Pull Request merge) for critical documentation, and record each decision with approver, role, timestamp and reason (LD-239, LD-251).

13	CI/CD Verification	GitHub Actions workflow	When a Pull Request is created or updated, run documentation drift analysis and automated tests, report the verification result, and handle workflow failures (LD-376).
14	Reliability	Background job handling	Retry failed AI processing jobs automatically, move jobs that keep failing to a Dead Letter Queue for later recovery, and keep processing events traceable through audit logs.
15	Audit	Audit logging	Record every administrative operation and apply the configured retention to the change log and the audit log (LD-280).
16	Deployment	Container deployment	Run the backend, frontend, AI service and Code Analyzer as Docker services deployed through the CI/CD pipeline (LD-382, LD-397).

3.1.13. Entity Relationship Diagra



4. Non-Functional Requirements

4.1 External Interfaces (in progress)

- GitHub account authorization and repository integration.
- Interfaces between the frontend, application services, AI service, and data storage.
- CI/CD reporting and deployment interfaces.

4.2 Quality Attributes

Attribute	Requirement status
Security and access control	User/role management and repository access are in scope; detailed security controls and measurable targets need confirmation.
Performance	Response-time and throughput targets are not specified in the reviewed Jira items.
Reliability	Retry, recovery, and availability targets need confirmation.
Maintainability	Component-based workstreams and automated checks are tracked; coding standards are not fully specified in Jira.

Usability	Frontend interfaces are tracked; formal usability criteria are not specified in the reviewed items.
Auditability	Approval audit trail and system audit/monitoring are in scope.
Scalability	Measurable scalability targets are not specified in the reviewed items.
Compatibility	Supported browsers, repository sizes, programming languages, and file formats need confirmation.

5. Requirement Appendix

5.1 Business Rules

ID	Rule Definition
BR-01	Each account has exactly one email address and one username, and neither can be used by another account. Email addresses typed in the registration and sign-in forms are converted to lower case, so two addresses that differ only in letter case identify the same account; an email received from a GitHub sign-in is stored exactly as GitHub returns it.
BR-02	Registration requires a username, an email address, a password, a first name and a last name. The username must have 3–50 characters, the password 6–255 characters and the email address at most 255 characters.
BR-03	Passwords are stored only as BCrypt hashes and are never stored or returned in plain text.
BR-04	Every new account, whether registered with the form or created by the first GitHub sign-in, receives the Developer role. The Staff, Manager, Technical Lead and Administrator roles can be granted only by an Administrator.
BR-05	A user signs in with email and password or with a GitHub account through OAuth. A failed sign-in shows the same message whether the email or the password is wrong.

BR-06	The first GitHub sign-in creates a LivingDocs account. If GitHub returns no public email, the GitHub no-reply address is used; if the email already belongs to another account, the sign-in is refused.
BR-07	Except registration, sign-in, sign-out, the CSRF token and health checks, every function requires an authenticated session, and every state-changing request from the browser must carry a valid CSRF token.
BR-08	When the session ends (sign-out or expiry), the web application returns to the sign-in page and the user must sign in again.
BR-09	A user can view and change only their own profile and notification preferences.
BR-10	A user has at most one GitHub connection; connecting again replaces the previous connection.
BR-11	A GitHub authorization request is valid for 10 minutes and can be used only once. An expired or reused request is rejected and the user must start the connection again.
BR-12	GitHub access and refresh tokens are encrypted with AES-256-GCM before they are stored and are never sent to the browser.
BR-13	A GitHub token that expires within 5 minutes is refreshed before it is used. If the refresh fails, the connection becomes Invalid and the user is asked to reconnect GitHub.
BR-14	Disconnecting GitHub revokes the authorization on GitHub, deletes the stored tokens, sets the connection to Revoked and sets its repositories to Disconnected.
BR-15	Each user works in a personal workspace (ws-{userId}) and can view or change only the repositories in that workspace.
BR-16	Only repositories that the LivingDocs GitHub App installation is allowed to access can be added to a workspace.
BR-17	A repository can be linked to a workspace only once; adding it again changes nothing.

BR-18	Removing a repository from a workspace deletes only the link in LivingDocs; nothing changes on GitHub.
BR-19	A connected repository is in one of four states: Connected, Syncing, Error or Disconnected. It changes to Error only when it can no longer be read (not found, access denied or invalid name); a temporary failure keeps the previous state.
BR-20	A synchronization reads the default branch unless another branch is chosen. It collects at most 200 source files by default (up to 1,000 on request) and the 20 most recent commits.
BR-21	Only Java and Python source files are collected for analysis. Files larger than 200,000 bytes and binary files are listed without content, symbolic links are skipped, and the rest of the repository is still synchronized.
BR-22	Files in tool and dependency folders (.git, .github, .idea, .vscode, .gradle, .mvn, node_modules, bower_components, venv, .venv, __pycache__, .pytest_cache, .tox, site-packages, .next, .nuxt) are always skipped. Files in build-output folders (target, build, dist, out, bin, obj, vendor, coverage) are skipped unless the folder is inside a src folder. Minified bundles (.min.js, .bundle.js) are skipped.
BR-23	A file whose path is empty or absolute, or contains "." or ".." segments, a backslash or a control character, is skipped with a warning; the synchronization continues with the other files.
BR-24	When the GitHub rate limit is reached, the user is told when to retry. Rate-limit and network failures can be retried; other failures require a user action (reconnect GitHub, grant repository access or choose another branch).
BR-25	Commits and Pull Requests are ingested only from repositories that are connected to the workspace.
BR-26	The code analyzer parses Python and Java files. Any other file is reported as Unsupported, an empty file as Empty and a file with syntax errors as Invalid; the remaining files are still analyzed.
BR-27	The web application generates documentation for the whole repository, from its latest successful synchronization, which

	must contain at least one source file with content. Generation for a single file or a module (package) is available only through the AI Service API.
BR-28	Each generation uses one template (default "Standard technical docs") and one output language, Vietnamese or English (default Vietnamese).
BR-29	Generated documentation may describe only behaviour that exists in the analyzed code. Every described element carries grounding evidence: the source file path, the code element, its line range and a short code excerpt.
BR-30	Each described element receives a confidence score from 0 to 1, rated High (≥ 0.75), Medium (≥ 0.50) or Low (< 0.50). The score of a job is the average of its elements.
BR-31	If the AI model fails three times for a file, that file is documented from the analysis data only and marked "fallback". Without an AI API key, the service runs in offline mode.
BR-32	A generation job can be viewed only by the user who started it. Its result and report are available only when the job is Completed (job statuses: Pending, Running, Completed, Failed).
BR-33	The web application waits up to 2 minutes for a generation job and then reports a time-out; the user can retry.
BR-34	The built-in templates (Standard technical docs, API reference, Brief summary) cannot be changed or deleted.
BR-35	Template names are unique, ignoring letter case. A name has at most 80 characters, a description 300 and the writing instructions 4,000. Sections are chosen without repetition from Overview, Classes, Functions and API Endpoints, and only elements of the chosen sections are written and scored.
BR-36	Deleting a custom template requires a second confirmation.
BR-37	A document cannot be saved with empty content.
BR-38	Restoring an earlier version requires confirmation because it replaces unsaved edits.

BR-39	Documentation is exported as a UTF-8 Markdown file named livingdocs-documentation.md.
BR-40	A Developer can preview and edit an AI-generated draft before submitting it; the draft enters the review and approval workflow only when the Developer submits it.
BR-41	A document has exactly one workflow status: Draft, In Review, Approved, Rejected or Published. Submission moves a Draft to In Review, final Manager approval moves it to Approved and publishing moves it to Published. A Staff or Manager rejection moves it to Rejected and returns it to the author with feedback.
BR-42	Every saved change (AI generation, AI auto-update or human edit) creates a new document version; existing versions are never overwritten.
BR-43	Each version is recorded in the change log with the acting role (AI, Staff or Manager), the timestamp and the trigger (commit, Pull Request or review action).
BR-44	Any two versions of a document can be compared in a difference view.
BR-45	A rollback restores the selected earlier version as a new version and records the rollback in the change log; no version is deleted.
BR-46	The change history is complete and tamper-evident: its entries cannot be edited or deleted and are kept for the retention period set by the Administrator.
BR-47	When source code changes in a commit or Pull Request, the affected documentation is updated automatically as a new version. Unless a Manager has configured that documentation type for auto-apply, the update requires Staff review and Manager approval before it is published.
BR-48	Automatic update can be enabled or disabled per repository, module or document. The Developer can accept, edit or dismiss an auto-updated version before it enters the approval workflow.
BR-49	Every documentation drift finding is classified as Referential, Signature or Semantic and given a severity of Low, Medium,

	High or Critical. It is resolved by Accept, Edit or Dismiss, and the outcome is kept in the drift history.
BR-50	AI suggestions (automatic updates and drift fixes) are never applied without human review. Each suggestion carries grounding evidence (code diff, commit and related ticket) and a confidence score.
BR-51	Knowledge-base search returns only documentation and code entities from repositories that the user can access.
BR-52	Submitted documentation is reviewed first by Staff. Only documents approved at the review stage reach the Manager's queue for final approval.
BR-53	Rejecting a document or requesting changes requires a reason; Staff can comment on specific sections.
BR-54	Documentation is published or committed to the repository only after Manager approval. The Manager decides which documentation types require approval and whether a Pull Request merge is warned or blocked for critical documentation.
BR-55	Every approval decision records the approver, the role, the time and the reason in the approval audit trail. A Manager can delegate approval to another Manager.
BR-56	LivingDocs has five roles: Developer, Staff, Manager, Technical Lead / Documentation Owner and Administrator. A user can use only the functions permitted to their role.
BR-57	The Technical Lead / Documentation Owner manages documentation collections and their owners, documentation templates, drift-detection policies and thresholds, and automatic-update policies.
BR-58	Only an Administrator manages users, roles and permissions, workspaces, repository access and system settings (GitHub App and webhooks, generation and update triggers, AI settings, log retention). Every administrative operation is recorded in the audit log.

5.2 Common Requirements

Req. ID	Requirement Name	Description
CR-01	Data Validation	Mandatory fields are checked in the form and again on the server. An invalid request is rejected with HTTP 400 and the user sees a short message that says what to correct.
CR-02	Data Format Validation	The system enforces these formats: • Email: standard email format, stored in lower case. • Repository: full name owner/repo; repository and connection IDs are UUIDs. • Date/Time: ISO-8601 with time zone. • Output language: vi (Vietnamese) or en (English). • Documents: UTF-8 Markdown.
CR-03	Session Security	The browser keeps only a session cookie and no access token. Every POST, PUT, PATCH and DELETE request carries the CSRF token in the X-XSRF-TOKEN header. Cookies use SameSite=Lax; SESSION_COOKIE_SECURE=true adds the Secure flag and is required when the system runs over HTTPS/TLS.
CR-04	Authorization	An API call without a valid session receives HTTP 401. Every request is checked for ownership: the workspace must be ws-{userId}, the repository must belong to the user's GitHub connection and a job must have been started by the user; otherwise the call receives HTTP 403. Role-based access control (RBAC) restricts functions to the five roles in BR-56.
CR-05	Internal Service Access	The browser calls only the Backend, through Nginx at /api. The Backend forwards GitHub, repository and documentation requests to the GitHub Integration Service with a shared

		internal credential and the user ID; a request without the credential receives HTTP 401. The Code Analyzer and the AI Service are reachable only inside the internal Docker network (and from the host on 127.0.0.1 for debugging).
CR-06	Secret Management	Secrets (GitHub client secret, token-encryption key, internal credential, database passwords, AI API key) are read from environment variables or a local .env file and are never committed; example files contain placeholders only. The GitHub Integration Service does not start without a 32-byte, Base64-encoded token-encryption key.
CR-07	Error Handling	The GitHub Integration Service returns errors as JSON {status, code, message, timestamp} and the Backend returns {message}. The interface shows that message, or a short replacement, and never a stack trace. GitHub failures are turned into the action the user must take: reconnect, grant access or retry later.
CR-08	Timeouts and Retries	Backend to GitHub Integration Service: connect 5 s, read 150 s. GitHub Integration Service to Code Analyzer and AI Service: connect 5 s, read 120 s. Browser requests stop after 180 s. Each AI call is tried up to 3 times (the first attempt and 2 retries) with back-off. When GitHub limits requests, the API returns a Retry-After value.
CR-09	Audit Logging	Services log operational events (user ID, GitHub account, error code, synchronization failures). Every administrative operation and every documentation decision is recorded with the user, role, time and action. Logs never contain tokens, passwords or source-file content.

CR-10	Data Integrity	Database constraints prevent duplicates: one email and one username per account, one GitHub connection per user and one link per repository–workspace pair. The GitHub Integration database schema is versioned with Flyway migrations.
CR-11	API Documentation and Health Checks	Service APIs are documented with OpenAPI: Swagger UI at /swagger-ui.html (GitHub Integration Service) and /docs (Python services). Health checks: /actuator/health (Spring Boot services), /health (AI Service) and /openapi.json (Code Analyzer).
CR-12	Language and Encoding	The interface is in English and generated documentation is in Vietnamese or English. All text uses UTF-8, so Vietnamese characters are preserved in API calls and exported files.
CR-13	Containerized Deployment	Each service runs in its own Docker container and is started with Docker Compose together with two PostgreSQL 16 databases. Every container except the web server has a health check, and a service starts only after the services it depends on are healthy.
CR-14	Continuous Integration	Every push to main or an LD-* branch and every Pull Request to main runs the CI pipeline: backend build and test, frontend build and lint, frontend end-to-end tests, GitHub Integration Service tests, AI Service tests, AI Service integration tests and Docker Compose validation. A Pull Request needs at least one approval before it is merged into main.
CR-15	AI Output Quality	Every AI-generated document or suggestion includes grounding evidence and a

		confidence score, and nothing produced by AI is published without human review.

5.3 Application Messages List

#	Code	Message Type	Context	Content
1	MSG 01	Inline (Success)	Sign-in succeeded	Signed in successfully as {email}.
2	MSG 02	Inline (Red)	Sign-in: wrong email or password (HTTP 401)	Invalid email or password.
3	MSG 03	Inline (Red)	Sign-in: invalid input (HTTP 400)	Please enter a valid email and password.
4	MSG 04	Inline (Red)	Sign-in: other failure	Login failed. Please try again.
5	MSG 05	Inline (Red)	Session expired (HTTP 401)	Your LivingDocs session has expired. Please sign in again.
6	MSG 06	Inline (Red)	Sign-in: Backend unreachable	Could not connect to the backend. Check the backend URL and try again.
7	MSG 07	Inline (Red)	Sign-out failed	Could not sign out. Please retry.
8	MSG 08	Inline (Red)	Sign-out: Backend unreachable	Could not connect to the backend to end the session.
9	MSG 09	API error	GitHub sign-in with an email that belongs to another account	This email already belongs to another account. Sign in with that account first.
10	MSG10	Inline (Red)	Registration: passwords differ	Passwords do not match.

11	MSG11	Inline (Success)	Registration succeeded	Registration successful.
12	MSG12	Inline (Red)	Registration: account exists (HTTP 409)	That username or email is already registered.
13	MSG13	Inline (Red)	Registration: invalid data (HTTP 400)	Please check the registration details and try again.
14	MSG14	Inline (Red)	Registration: other failure	Registration failed. Please try again.
15	MSG15	Inline (Red)	Registration: Backend unreachable	Could not connect to the backend. Check that it is running and try again.
16	MSG16	API error (400)	Mandatory field missing	{Field} is required
17	MSG17	API error (400)	Field length	Username must be between 3 and 50 characters / Password must be between 6 and 255 characters / Email must not exceed 255 characters
18	MSG18	API error (400)	Email format	Email must be valid
19	MSG19	Inline (Red)	Connect GitHub without signing in	Sign in to LivingDocs before connecting GitHub.
20	MSG 20	Inline (Success)	GitHub connected (the second form is used when GitHub returns no login name)	GitHub connected as @{login}. / GitHub connected.
21	MSG 21	Inline (Red)	GitHub connection failed (callback)	GitHub connection failed ({reason}).

22	MSG 22	Inline (Red)	Start GitHub authorization failed	Could not start GitHub authorization.
23	MSG 23	Inline (Red)	No authorization URL returned	GitHub did not return an authorization URL.
24	MSG 24	Inline (Success)	GitHub disconnected	GitHub disconnected.
25	MSG 25	Inline (Red)	Disconnect GitHub failed	Could not disconnect GitHub.
26	MSG 26	API error (400)	User denied access on GitHub (GITHUB_ACCESS_DENIED)	Bạn chưa cho phép LivingDocs truy cập GitHub. Hãy bấm 'Kết nối GitHub' lại nếu muốn kết nối.
27	MSG 27	API error (400)	Authorization request expired or reused (BAD_REQUEST)	State không hợp lệ hoặc đã hết hạn. Hãy bấm 'Kết nối GitHub' lại từ đầu.
28	MSG 28	API error (409)	No active GitHub connection (GITHUB_NOT_CONNECTED)	Bạn chưa kết nối GitHub hoặc kết nối đã bị ngắt. Hãy kết nối lại.
29	MSG 29	API error (401)	GitHub token expired or revoked (GITHUB_REAUTH_REQUIRED)	Token GitHub không còn hợp lệ. Hãy kết nối lại GitHub.
30	MSG 30	API error (502)	GitHub rate limit (GITHUB_API_ERROR)	GitHub API rate limit exceeded. Please retry later.
31	MSG 31	API error (502)	GitHub unreachable (GITHUB_API_ERROR)	Unable to contact GitHub. Please retry later.

32	MSG 32	Inline (Info)	Repository page: connection status	Connected to GitHub. / GitHub is not connected.
33	MSG 33	Inline (Red)	Repository page without signing in	Sign in to LivingDocs to load your GitHub repositories.
34	MSG 34	Inline (Info)	Empty repository list	Connect GitHub to view repositories.
35	MSG 35	Inline (Red)	Load repositories failed	Could not load repositories.
36	MSG 36	Inline (Red)	Add with no repository selected	Select at least one repository.
37	MSG 37	Inline (Success)	Repositories added	Selected repositories added to workspace.
38	MSG 38	Inline (Red)	Add repositories failed	Could not add repositories to the workspace.
39	MSG 39	Inline (Success)	Repository synchronized	{owner/repo} synchronized.
40	MSG 40	Inline (Red)	Synchronization failed	Repository synchronization failed.
41	MSG 41	Inline (Success)	Repository removed	{owner/repo} removed from workspace.
42	MSG 42	Inline (Red)	Remove repository failed	Could not remove repository.
43	MSG 43	API error (400)	Repository outside the GitHub App installation (BAD_REQUEST)	Repository {id} không nằm trong danh sách repo mà GitHub App được phép truy cập.
44	MSG 44	API error (403)	Repository of another user (FORBIDDEN)	Repository này không thuộc kết nối GitHub của bạn.

45	MSG 45	API error (403)	Workspace of another user (FORBIDDEN)	Workspace này không thuộc người dùng đã xác thực.
46	MSG 46	API error (429)	Sync: GitHub rate limit (RATE_LIMITED)	Đã vượt giới hạn số lần gọi GitHub API, thử lại sau {time}. / Đã vượt giới hạn số lần gọi GitHub API, thử lại sau ít phút.
47	MSG 47	API error (403)	Sync: GitHub App cannot read the repository (ACCESS_DENIED)	GitHub App không có quyền đọc repository {repo}. Hãy cài app cho repo này và cấp quyền Contents (read).
48	MSG 48	API error (404)	Sync: repository not found (REPOSITORY_NOT_FOUND)	Không tìm thấy repository {repo} (repo đã bị xoá, đổi tên, hoặc GitHub App chưa được cài cho repo này).
49	MSG 49	API error (404)	Sync: branch not found (BRANCH_NOT_FOUND)	Không tìm thấy nhánh '{branch}' trong repository {repo}.
50	MSG 50	API error (409)	Sync: repository has no commits (EMPTY_REPOSITORY)	Repository {repo} chưa có commit nào nên không có gì để đồng bộ.
51	MSG 51	API error (503)	Sync: network failure or time-out (GITHUB_UNAVAILABLE)	Không kết nối được tới GitHub (mất mạng hoặc quá thời gian chờ). Thử lại sau.
52	MSG 52	API error (404)	Analysis input requested before any sync (NOT_FOUND)	Repository này chưa được đồng bộ lần nào (hoặc service vừa khởi động lại). Hãy gọi POST /api/repositories/{id}/sync trước.

53	MSG 53	Inline (Red)	Generate without signing in	Sign in to LivingDocs before generating documentation.
54	MSG 54	Inline (Red)	Workspace has no repository	Add a repository to your workspace before generating documentation.
55	MSG 55	Inline (Red)	No repository selected	Select a workspace repository before generating documentation.
56	MSG 56	Status label	Generation status	Ready to generate / Generating... / Generation completed / Generation failed
57	MSG 57	Inline (Success)	Documentation generated	Documentation generated from {owner/repo}.
58	MSG 58	Inline (Red)	Generation took longer than 2 minutes	Documentation generation timed out. You can retry from this page.
59	MSG 59	Inline (Red)	Generation failed	Documentation generation failed.
60	MSG 60	API error (400)	Generate: synchronized files have no content (BAD_REQUEST)	The latest repository sync contains no source files with content.
61	MSG 61	API error (502)	Code Analyzer failed (ANALYZER_FAILED)	The code analyzer could not analyze the synchronized repository.
62	MSG 62	API error (502)	AI Service failed (AI_SERVICE_FAILED)	The AI service could not start documentation generation.

63	MSG 63	API error (409)	Result requested before the job finished (DOCUMENTATION_NOT_READY)	The documentation result is not ready yet.
64	MSG 64	API error (404)	Job not found after a service restart (NOT_FOUND)	Documentation job not found or the service was restarted.
65	MSG 65	API error (403)	Job of another user (FORBIDDEN)	This documentation job belongs to another user.
66	MSG 66	Inline (Red)	Save empty content	Document content cannot be empty.
67	MSG 67	Inline (Success)	Draft saved	Draft saved in this page session.
68	MSG 68	Status label	Editor status	All changes saved in the current page session. / You have unsaved edits.
69	MSG 69	Inline (Red)	Create a version with no content	Generate or enter document content before creating a version.
70	MSG70	Inline (Success)	Version created	Version {n} created.
71	MSG71	Confirm dialog	Restore a version	Restore Version {n}? Current unsaved edits will be replaced.
72	MSG72	Inline (Success)	Version restored	Content restored from Version {n}.
73	MSG73	Inline (Red)	Restore with no version selected	Select a version to restore.
74	MSG74	Inline (Info)	Compare versions	Create a version first to enable comparison. /

				Select a version to display the comparison.
75	MSG75	API error (404)	Template not found (TEMPLATE_NOT_FOUND)	Documentation template not found.
76	MSG76	API error (403)	Change or delete a built-in template (TEMPLATE_READ_ONLY)	Built-in templates cannot be changed or deleted.
77	MSG 77	API error (409)	Duplicate template name (TEMPLATE_NAME_TAKEN)	A template with this name already exists.
78	MSG78	Inline (Red)	Request took longer than 180 seconds	The request timed out. Please try again.
79	MSG79	Inline (Red)	GitHub Integration Service unreachable	Could not connect to the GitHub integration service.
80	MSG 80	Inline (Red)	Unexpected HTTP error	Request failed (HTTP {status}).
81	MSG 81	Inline (Red)	CSRF token unavailable	Could not obtain a request security token.
82	MSG 82	API error (502)	Backend cannot reach the GitHub Integration Service	GitHub Integration is unavailable.
83	MSG 83	API error (503)	Internal credential not configured	GitHub Integration internal credential is not configured.
84	MSG 84	API error (401)	Call without the internal credential (INTERNAL_SERV	A trusted internal service credential is required.

			ICE_AUTH_REQUIRED)	
85	MSG 85	Inline (Red)	Save a template without a name or instructions	Enter a name and writing instructions for the template.
86	MSG 86	Inline (Red)	Save a template with no section	Choose at least one section.
87	MSG 87	Inline (Success)	Template saved	Template "{name}" saved.
88	MSG 88	Inline (Red)	Save template failed	Could not save the template.
89	MSG 89	Inline (Success)	Template deleted (after "Confirm delete")	Template "{name}" deleted.
90	MSG 90	Inline (Red)	Delete template failed	Could not delete the template.
91	MSG 91	Inline (Red)	Template list unavailable	Could not load templates from the server. Showing the built-in templates.
92	MSG 92	API error (400)	Generate with a template that no longer exists (TEMPLATE_NOT_FOUND)	The selected documentation template does not exist. Choose another template.
93	MSG 93	Inline (Red)	Generate: the service returned a job without an ID	The documentation service returned an invalid job.
94	MSG 94	Inline (Red)	Generate: the result is empty	The documentation service returned an empty document.
95	MSG 95	Inline (Red)	CSRF token response is invalid	The backend returned an invalid request security token.

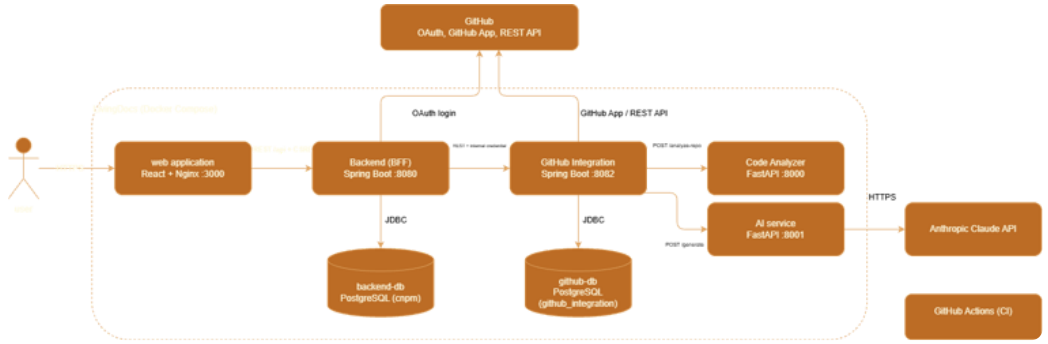
96	MSG 96	Inline (Success)	Profile saved (demo screen, no API call yet)	Profile updated in this demo. Backend is not connected.
97	MSG 97	Inline (Success)	Notification preferences saved (demo screen, no API call yet)	Preferences saved successfully! (Demo only)
98	MSG 98	API error (502)	AI Service returned a job without an ID (AI_SERVICE_INVALID_RESPONSE)	The AI service returned an invalid job response.
99	MSG 99	API error (502)	AI Service returned an empty result (AI_SERVICE_INVALID_RESPONSE)	The AI service returned an empty documentation result.
100	MSG100	API error (404)	Result or report missing in the AI Service (NOT_FOUND)	Documentation job or result not found in the AI service. / Documentation job or report not found in the AI service.
101	MSG101	API error (400)	Generation settings rejected by the AI Service (GENERATION_REQUEST_INVALID)	The documentation generation settings are not valid.

Message types: Inline messages are shown on the page, in green (Success), red (Red) or blue (Info). A Status label shows the state of a running action, and a Confirm dialog asks before edits are replaced. API errors are JSON responses from the services; the web application shows them on the page. Text in braces is replaced at run time. MSG26–MSG29 and MSG43–MSG52 are returned in Vietnamese by the GitHub Integration Service.

IV. Software Design Description

1. System Design

1.1 System Architecture



1.1 System Architecture

The overall system architecture of **LivingDocs** is illustrated in Figure 1.1. The system follows a **microservice architecture**: a React web application, a Spring Boot Backend that is the only entry point for the browser (Backend-for-Frontend), a Spring Boot GitHub Integration Service and two Python FastAPI services (Code Analyzer and AI Service), with two PostgreSQL 16 databases. Every part runs as a Docker container, and the services communicate through REST APIs over three internal networks: backend-network, github-internal and analysis-internal.

1. UI Components

- **Web Application (React 19 + Vite 8, served by Nginx on port 3000):** one single-page application for every role.
  - **Functionality:** sign-in and registration, profile and notification preferences, GitHub connection, repository management (add, synchronize, remove) and documentation management (choose a template, generate, preview, edit, create and compare versions, export Markdown).
  - **Note:** the browser never calls an internal service directly; every request goes to the Backend under /api. Screens for the Staff review queue, the Manager approval queue, Technical Lead governance and Administrator settings are planned.

2. Core System (Server-side)

- **Backend – Backend-for-Frontend (Spring Boot 4.1, Java 21, port 8080):** manages accounts, form and GitHub OAuth sign-in, sessions and CSRF protection, and forwards GitHub, workspace, repository and documentation requests to the GitHub Integration Service with an internal credential and the user ID.
- **GitHub Integration Service (Spring Boot 3.5, Java 21, port 8082):** connects each user's GitHub account through the GitHub App (OAuth authorization, encrypted tokens), manages workspace repositories and their synchronization, and orchestrates documentation generation and template requests.
- **Code Analyzer (Python FastAPI, port 8000):** parses Python (ast) and Java (javalang) source files and extracts classes, functions and API endpoints.

- **AI Service (Python FastAPI, port 8001):** runs documentation generation jobs (evidence catalogue, retrieval of related code and documentation, LLM call with up to three attempts, output validation, Markdown rendering and confidence scoring) and stores documentation templates.

3. **External Systems / Third-parties**

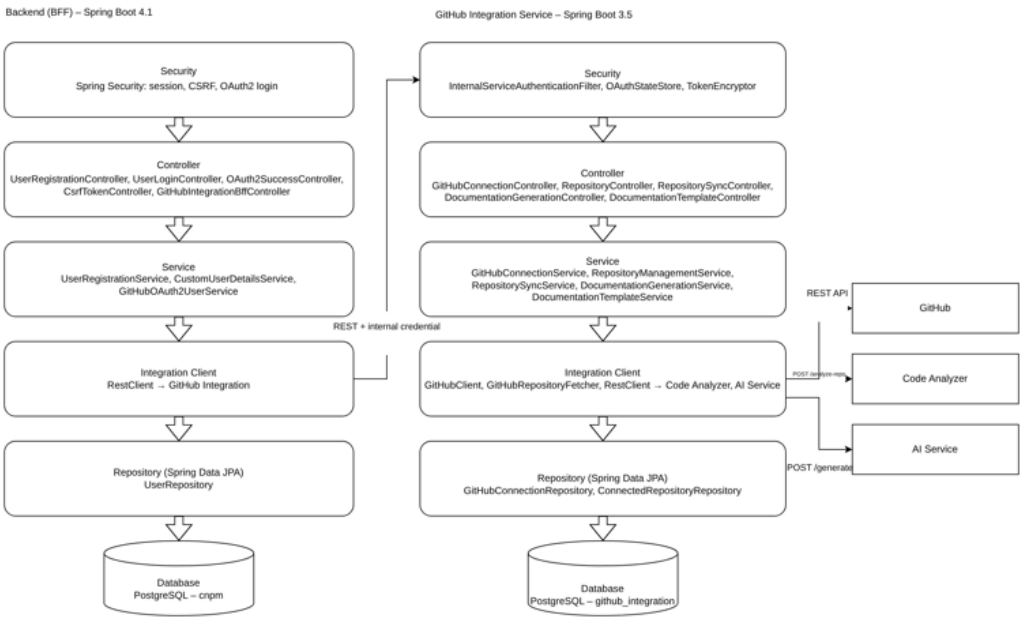
- **GitHub:** OAuth sign-in (scopes read:user and user:email), GitHub App authorization and the REST API for repositories, file trees, file contents and commits.
- **Anthropic Claude API:** the large language model that writes the documentation text. It is optional: without an API key, the AI Service builds documentation from the analysis data only (offline mode).
- **Docker / Docker Compose:** package and run every service and database in its own container.
- **GitHub Actions:** the CI pipeline (seven jobs) that builds and tests every push to main or an LD-* branch and every Pull Request to main.
- **Planned:** GitHub webhooks for Pull Request events (automatic update and drift detection), a Kafka/RabbitMQ message broker for asynchronous AI jobs, and Redis for caching.

4. **Database Layer**

- **PostgreSQL 16 – backend-db (database cnpm):** user accounts and their roles (table users).
- **PostgreSQL 16 – github-db (database github_integration):** GitHub connections and connected repositories (tables github_connections and repositories); the schema is versioned with Flyway.
- **Template store:** custom documentation templates are saved as a JSON file on the Docker volume ai-templates, so they survive restarts.
- **In-memory data (current release):** the latest synchronization snapshot of each repository and the generation jobs are kept in memory and are lost when a service restarts.
- **Planned:** document versions, change log, review decisions and audit logs stored in PostgreSQL.

1.1.1 Backend Architecture

The two Spring Boot services (Backend and GitHub Integration Service) follow a layered architecture pattern to ensure separation of concerns and maintainability. Each request passes through the security layer, a controller, a service and a repository before it reaches the database.

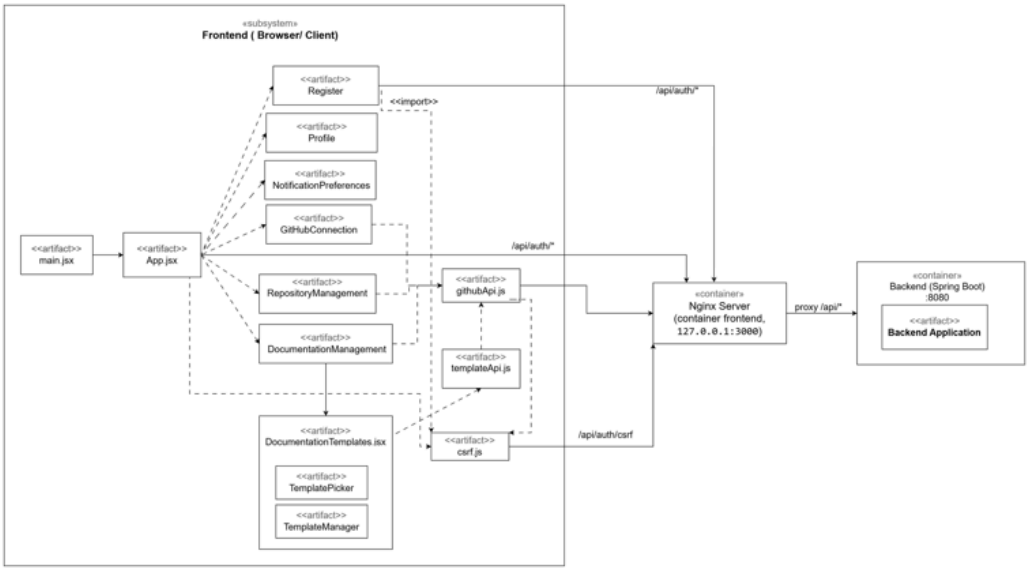


1.1.1 Backend Architecture

- **Security Layer:** in the Backend, the Spring Security filter chain handles sessions, CSRF tokens and OAuth2 login and returns HTTP 401 for unauthenticated /api/** calls. In the GitHub Integration Service, InternalServiceAuthenticationFilter accepts only requests that carry the internal credential (except the OAuth callback and the health check).
- **Controller Layer:** the entry point for HTTP requests. It validates input and delegates to a service (for example UserRegistrationController, GitHubIntegrationBffController, RepositorySyncController and DocumentationGenerationController).
- **Service Layer (Business Logic):** applies the business rules of section 5.1, such as ownership checks, token refresh, synchronization limits and job ownership (for example GitHubConnectionService, RepositorySyncService and DocumentationGenerationService).
- **Integration Clients:** the Backend calls the GitHub Integration Service with RestClient. The GitHub Integration Service calls GitHub through GitHubClient and GitHubRepositoryFetcher, and calls the Code Analyzer and the AI Service through RestClient.
- **Repository Layer (Spring Data JPA / Hibernate):** reads and writes entities (UserRepository, GitHubConnectionRepository and ConnectedRepositoryRepository).
- **Database (PostgreSQL):** the persistent storage of each service (cnpm and github_integration).
- **Error Handling:** GlobalExceptionHandler converts exceptions into JSON {status, code, message, timestamp} in the GitHub Integration Service; Backend controllers return {message}.

1.1.2 Web Application Architecture

The Web Application is a **React 19** single-page application built with **Vite 8** and served by **Nginx**, which also forwards /api/ requests to the Backend.



1.1.2 Web Application Architecture

- **Entry Point (main.jsx):** mounts the App component into the page.
- **App Shell (App.jsx):** keeps the current page in React state instead of a URL router, stores the signed-in user ID in sessionStorage, handles sign-in and sign-out, and returns to the sign-in page when a livingdocs:unauthenticated event is raised.
- **Pages:** Register, Profile, NotificationPreferences, GitHubConnection, RepositoryManagement and DocumentationManagement (which embeds TemplatePicker and TemplateManager).
- **Data Layer:** githubApi.js, templateApi.js and csrf.js call the REST API with fetch, the session cookie and the X-XSRF-TOKEN header, stop a request after 180 seconds and turn HTTP errors into user-friendly messages.
- **Web Server (Nginx):** serves the built static files and proxies /api/ to backend:8080.

1.1.3 Analysis and AI Services Architecture

Documentation is produced by a fixed pipeline: synchronize, analyze, generate, then poll for the result.

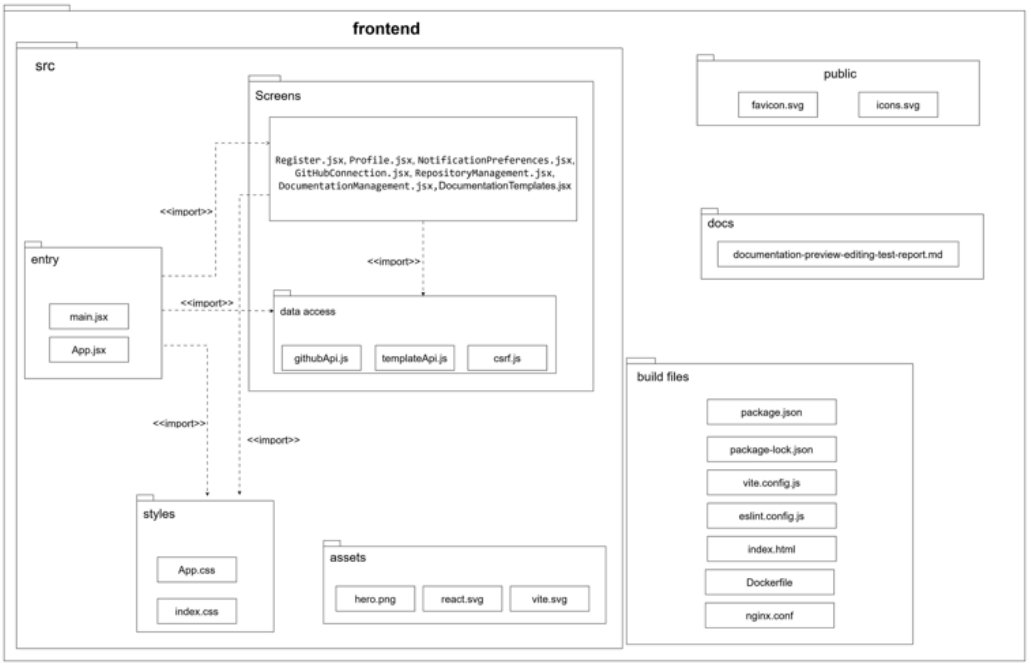
1. The user clicks Generate documentation. The browser calls POST /api/repositories/{id}/documentation/generate, which the Backend forwards to the GitHub Integration Service.
2. The GitHub Integration Service takes the latest synchronization snapshot of the repository and builds the analysis input (scope, target and files).

3. The Code Analyzer detects the language of each file, parses it (Python ast, Java javalang) and extracts classes, functions and endpoints.
4. The GitHub Integration Service sends the analysis result to the AI Service and receives a job in Pending status (HTTP 202).
5. The AI Service processes the files one by one: evidence catalogue, prompt, Claude API call (up to three attempts), JSON validation, Markdown rendering and confidence scoring.
6. The browser polls the job status every 750 ms for up to 2 minutes, then downloads the Markdown result and the evidence and confidence report.

1.2 Package Diagram

This section describes the package structure of the LivingDocs applications. The source code is divided into three groups: the Web Application (frontend/), the Java backend services (backend/ and services/github-integration-service/) and the Python services (app/ and code-analyzer/ for the Code Analyzer, ai-service/ for the AI Service). The Java services organize packages by layer, while the Python services use flat modules.

1.2.1 Web App Package Diagram



1.2.1 Web App Package Diagram

The Web Application follows a flat **React + Vite** project structure: all source code is in frontend/src, with one component file per screen and a small data-access layer.

1. Package Descriptions

The source code is organized into the following directories and file groups:

No	Package	Description

01	**src**	Application source: entry point (main.jsx), application shell (App.jsx), screens, data-access modules and styles.
02	**src – screens**	Register.jsx, Profile.jsx, NotificationPreferences.jsx, GitHubConnection.jsx, RepositoryManagement.jsx, DocumentationManagement.jsx and DocumentationTemplates.jsx. Profile.jsx and NotificationPreferences.jsx are demo screens that do not call the API yet.
03	**src – data access**	githubApi.js (REST client with the session cookie, CSRF header, 180-second time-out and error messages), templateApi.js (template API with a fallback to the built-in templates) and csrf.js (fetches the CSRF token from /api/auth/csrf).
04	**src – styles**	App.css and index.css: global and page styles.
05	**src/assets**	Images and icons used by the screens.
06	**public**	Static files served as they are (favicon and icon sprite).
07	**docs**	Frontend test report for documentation preview and editing.
08	**build and deployment files**	index.html, package.json, package-lock.json, vite.config.js, eslint.config.js, Dockerfile and nginx.conf: page shell, dependencies, build, lint, container image and web-server configuration.

1.2.1.1 Web App Package Description

2. Page Structure (App Shell Navigation)

Screens are selected by a page key held in App.jsx rather than by URL routes:

No	Page Key	Component	Description

01	**login**	App	Sign-in form (email and password) with links to registration, GitHub connection, profile and notification pages.
02	**register**	Register	Create an account.
03	**profile**	Profile	View and edit the user profile (demo, no API call yet).
04	**notifications**	NotificationPreferences	Choose which notifications to receive (demo, no API call yet).
05	**github**	GitHubConnection	Connect or disconnect GitHub; also opened when GitHub redirects back (?github=...).
06	**repository**	RepositoryManagement	Manage the repositories of the workspace.
07	**documentation**	DocumentationManagement	Generate and edit documentation; choose and manage templates (Manage templates).

1.2.1.2 Web App Page Structure Description

All current pages serve the Developer role. Review, approval, governance and administration pages for the Staff, Manager, Technical Lead and Administrator roles are planned and will be added as new page keys.

3. Components Structure

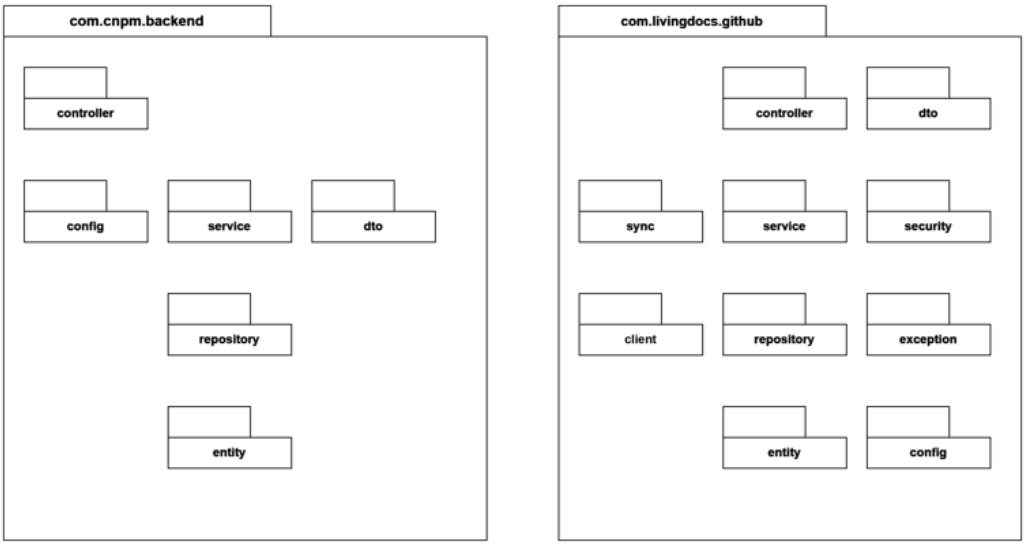
UI components are organized by screen and function:

No	Component Module	Description
01	**App**	Application shell: page switching, sign-in and sign-out, session-expiry handling.
02	**Register**	Registration form with password confirmation.

03	**Profile**	User profile form (full name and email); demo, not connected to the API yet.
04	**NotificationPreferences**	Notification switches: email updates, document changes, review requests and system notifications; demo, not connected to the API yet.
05	**GitHubConnection**	GitHub connection status, connect and disconnect.
06	**RepositoryManagement**	Available and workspace repositories: search, add, synchronize and remove.
07	**DocumentationManagement**	Documentation workspace: template selection, generation, preview, editing, versions, comparison and Markdown export.
08	**TemplatePicker**	Template selector used before generation (DocumentationTemplates.jsx).
09	**TemplateManager**	Create, edit and delete custom templates (DocumentationTemplates.jsx).

1.2.1.3 Web App Components Structure Description

1.2.2 Backend Package Diagram



1.2.2 Backend Package Diagram

The backend is built with **Spring Boot** on Java 21 (Backend 4.1, GitHub Integration Service 3.5) and follows a **Layered Architecture** inside each microservice. This ensures a clean

separation between the Presentation (Controller), Business Logic (Service) and Data Access (Repository) layers.

1. Source Structure Descriptions

Backend – base package: com.cnpm.backend

No	Package	Description
01	**config**	SecurityConfig (filter chain, CORS, CSRF, BCrypt), SpaCsrfTokenRequestHandler and RestClientConfig.
02	**controller**	Registration, sign-in and sign-out, GitHub sign-in success, CSRF token, and GitHubIntegrationBffController, which forwards requests to the GitHub Integration Service.
03	**service**	UserRegistrationService, CustomUserDetailsService and GitHubOAuth2UserService.
04	**repository**	UserRepository (Spring Data JPA).
05	**entity**	User, mapped to the users table.
06	**dto**	LoginRequest and RegisterRequest with validation rules.

1.2.2.1 Backend Source Structure Descriptions

GitHub Integration Service – base package: com.livingdocs.github

No	Package	Description
01	**config**	GitHubProperties, CorsConfig, OpenApiConfig and DocumentationServiceConfig (clients for the Code Analyzer and the AI Service).
02	**security**	InternalServiceAuthenticationFilter, OAuthStateStore (single-use state valid for 10 minutes), TokenEncryptor and EncryptedStringConverter (AES-256-GCM).

03	**controller**	GitHubConnectionController, RepositoryController, RepositorySyncController, DocumentationGenerationController and DocumentationTemplateController.
04	**service**	GitHubConnectionService, RepositoryManagementService, RepositorySyncService, DocumentationGenerationService and DocumentationTemplateService.
05	**sync**	Synchronization engine: GitHubRepositoryFetcher, PathFilter, LanguageDetector, SnapshotVerifier, SyncOptions, SyncErrorCode, RepositorySnapshot and AnalysisInput.
06	**client**	GitHubClient and the GitHub data records (GitHubRepo, GitHubTree, GitHubBlob, GitHubCommit, GitHubUser, GitHubTokenResponse).
07	**repository**	GitHubConnectionRepository and ConnectedRepositoryRepository (Spring Data JPA).
08	**entity**	GitHubConnection, ConnectedRepository, ConnectionStatus and RepositoryStatus.
09	**dto**	Request and response records, including DocumentationTemplateRequest and ErrorResponse.
10	**exception**	ApiException, GitHubApiException and GlobalExceptionHandler.
11	**db/migration (resources)**	Flyway script V1__init_github_integration.sql.

1.2.2.2 GitHub Integration Service Source Structure Descriptions

2. Business Modules Structure

The application core is divided into business modules, each with its own controller, service and data-access classes:

No	Module	Description
01	**Account & Authentication**	Registration, form sign-in, GitHub sign-in and sign-out (UserRegistrationController, UserLoginController, OAuth2SuccessController, GitHubOAuth2UserService).
02	**GitHub Connection**	Connect, refresh tokens and disconnect (GitHubConnectionController, GitHubConnectionService, OAuthStateStore, TokenEncryptor).
03	**Repository Management**	Available repositories; add, list and remove workspace repositories (RepositoryController, RepositoryManagementService).
04	**Repository Synchronization**	Read files and commits with limits and filters (RepositorySyncController, RepositorySyncService, GitHubRepositoryFetcher).
05	**Documentation Generation**	Call the analyzer, create AI jobs, check job ownership and return results and reports (DocumentationGenerationController, DocumentationGenerationService).
06	**Documentation Templates**	Forward template requests (list, create, update, delete) to the AI Service (DocumentationTemplateController, DocumentationTemplateService).

1.2.2.3 Backend Business Modules Structure

Planned modules for version history and change log, review and approval, drift detection and administration will follow the same structure.

3. Module Internal Structure (Common Pattern)

Each business module follows this internal layering:

No	Package	Description
01	**controller**	REST controllers (@RestController) that receive HTTP requests and return responses.
02	**service**	Business logic; this layer enforces the business rules.
03	**repository**	Data access layer; extends JpaRepository to read and write PostgreSQL.
04	**entity**	JPA entities (@Entity) mapped to database tables, with their status enumerations.
05	**dto**	Request and response records with validation annotations, decoupled from the entities.
06	**exception**	Custom exceptions and the global handler (@RestControllerAdvice) that returns JSON errors.
07	**config / security**	Spring configuration beans, CORS, security filters and token encryption.

1.2.2.4 Backend Module Internal Structure Description

4. Key Services Description

The following core services orchestrate the main business logic:

No	Service	Description
01	**UserRegistrationService**	Checks that the username and email are unique, hashes the password and assigns the Developer role.
02	**GitHubOAuth2UserService**	Finds or creates the LivingDocs account when a user signs in with GitHub.
03	**GitHubIntegrationBffController**	Forwards requests with the internal credential and user ID; returns HTTP 502 when the GitHub Integration Service is unreachable and HTTP 503 when the internal credential is not configured.

04	**GitHubConnectionServic e**	Builds the authorization URL, handles the callback, and refreshes and revokes tokens.
05	**RepositoryManagementS ervice**	Lists accessible repositories and checks workspace ownership.
06	**RepositorySyncService**	Runs synchronization, updates the repository status and keeps the latest snapshot.
07	**DocumentationGeneratio nService**	Sends the snapshot to the Code Analyzer, creates the AI job and checks job ownership.
08	**DocumentationTemplate Service**	Forwards template operations to the AI Service and maps its errors.

1.2.2.5 Backend Key Services Description

1.2.3 Python Services Package Diagram

The Code Analyzer and the AI Service are small **FastAPI** applications; each module performs one step of the pipeline.

No	Module	Description
01	**app/main.py**	Code Analyzer API: POST /api/v1/analyze-repo.
02	**app/models.py**	Request and result models (FileInput, FileAnalysis, ClassInfo, Function, Endpoint).
03	**app/reader.py**	Detects the language from the file extension.
04	**app/analyzer.py**	Parses Python (ast) and Java (javalang) into syntax trees.
05	**app/extractor.py**	Extracts classes, functions and endpoints.
06	**app/dependency.py**	Lists Python and Java imports (not used by the current pipeline).

07	**ai-service/main.py**	AI Service API: /generate, /jobs, /jobs/{id}/result, /jobs/{id}/report, /templates and /health.
08	**ai-service/app/jobs.py**	In-memory job store and background worker.
09	**ai-service/app/generator.py**	Calls the Claude API with retries; fallback and offline modes.
10	**ai-service/app/prompts.py, doc_schema.py**	Builds prompts and validates the JSON returned by the model.
11	**ai-service/app/evidence.py, retrieval.py**	Evidence catalogue and TF-IDF retrieval of related code and documentation.
12	**ai-service/app/confidence.py**	Confidence score and level for each element.
13	**ai-service/app/renderer.py**	Renders Markdown for each file and for the overview.
14	**ai-service/app/templates.py, models.py**	Template store (three built-in templates and custom templates saved to TEMPLATES_FILE) and shared data models.

1.2.3.1 Python Services Module Description

2. Database Design

A complete ERD and data dictionary are not specified by the reviewed Jira issue summaries. The backlog does identify user/role management, repositories, documentation, versions, review/approval, audit, and job-related capabilities. These suggest data domains that need to be represented in the final design, but the exact entities, columns, relationships, constraints, and indexes must be verified against the implementation.

Data domain	Jira traceability	Design evidence required
Users and roles	LD-15–LD-20, LD-257, LD-261	Entity definitions, role relationships, unique constraints,

		password/authentication policy
GitHub accounts and repositories	LD-22–LD-26, LD-267	Connection records, repository identifiers, access scope, synchronization state
Documentation and templates	LD-80–LD-83, LD-96, LD-204, LD-223	Document metadata, content storage, template schema, ownership and permissions
Generation jobs and AI results	LD-106, LD-340, LD-352	Job state, output references, evidence/confidence data, failure details
Source changes and drift	LD-112–LD-167, LD-346	Change references, affected-document links, drift classification and history
Documentation versions	LD-173–LD-195	Version identifiers, diff data, retention, rollback relationships
Review and approval	LD-229–LD-251	Submission state, comments, decisions, timestamps, audit history
System monitoring	LD-260, LD-280	Events, notifications, job logs, retention and access policy

3. Detailed Design

The template's detailed design chapter is adapted to the main LivingDocs workflows. These are behavioral summaries from Jira requirements, not implementation-level sequence or class diagrams.

3.1 Registration and Authentication

Related Jira: LD-15, LD-16, LD-17, LD-20, LD-38, LD-43.

1. A user provides registration information or selects an available sign-in method.
2. The system validates the request and applies authentication rules.
3. The system reports success or an appropriate failure.
4. Protected functions require the appropriate authenticated access.

Exact request fields, session/token design, password policy, and response codes must be taken from the implementation and detailed Jira descriptions.

3.2 Connect and Synchronize a GitHub Repository

Related Jira: LD-22–LD-26, LD-53, LD-58, LD-63, LD-68, LD-74.

1. A user authorizes a GitHub account connection.
2. The system presents repositories available under the granted permissions.
3. The user selects or manages a repository.
4. The system synchronizes supported repository information and ingests relevant commit/pull-request information.
5. The system reports connection or synchronization problems.

The exact webhook/polling mechanism, OAuth scopes, and retry rules are not specified in the reviewed issue summaries.

3.3 Generate Documentation

Related Jira: LD-79–LD-106, LD-328, LD-340, LD-352.

1. A user selects project/source information and requests documentation generation.
2. The source-analysis component processes supported code.
3. The AI service generates documentation using available project context.
4. The system exposes the generated result and any available evidence, confidence information, or job status.
5. The user previews/edits the content and may submit it for review.

Supported languages, prompt construction, output formats, and confidence calculation require confirmation.

3.4 Automatic Update and Drift Review

Related Jira: LD-112–LD-167, LD-346.

1. The system detects a supported source change.
2. It identifies documentation potentially affected by that change.
3. Drift analysis evaluates possible mismatches between source and documentation.
4. Findings may be classified and assigned a severity.

5. A user reviews findings and available suggestions.
6. The system records available history and supports the agreed update/review/rollback workflow.

The final detection algorithm, classification taxonomy, and severity thresholds are not specified in the reviewed issue summaries.

3.5 Version History and Rollback

Related Jira: LD-173–LD-195.

1. The system maintains available documentation versions.
2. A user views the change log or compares versions.
3. An authorized user selects an eligible version for rollback.
4. The system validates the request and records the result.

Version-retention policy and rollback conflict handling need to be defined.

3.6 Review and Approval

Related Jira: LD-229–LD-251.

1. A submitter sends documentation into the review workflow.
2. A staff reviewer examines the submission and may leave comments or request changes.
3. An authorized approver approves or rejects the item.
4. The system records available decisions and audit details.

The exact state machine, reassignment rules, and permissions should be documented from the implementation.

3.7 CI/CD and Deployment

Related Jira: LD-364, LD-370, LD-376, LD-382, LD-388.

1. A configured repository event triggers the pipeline.
2. The pipeline runs the configured build and test checks.
3. Results are reported for inspection.
4. Deployment follows the configured environment and approval rules.
5. System integration and acceptance tests validate the integrated system.

Trigger rules, required checks, environment variables, secrets, health checks, and production release policy need confirmation.

V. Software Testing Documentation

1. Scope of Testing

1.1 In-Scope Testing

The Jira backlog explicitly includes:

- Automated testing (LD-370).
- CI/CD pipeline checks (LD-364).
- GitHub Actions integration (LD-376).
- Docker and deployment validation (LD-382).
- System integration and acceptance testing (LD-388).

Testing should also cover the key functional areas in Section III, especially authentication, repository integration, documentation generation, drift/update workflows, versions, review/approval, and administration.

1.2 Out-of-Scope Testing

The reviewed Jira items do not define a formal out-of-scope testing list. Any exclusions must be approved and documented before final testing.

1.3 Constraints and Assumptions

- Jira task status is used for tracking and is not equivalent to a test pass.
- Test environments, test datasets, credentials, supported repository fixtures, and AI evaluation datasets are not fully specified in the reviewed issue summaries.
- Acceptance criteria must be linked to the relevant requirements before final sign-off.

2. Test Strategy

2.1 Testing Types

Test type	Purpose	Jira traceability
Automated tests	Validate configured behavior automatically.	LD-370
CI build checks	Validate that changes pass configured build checks.	LD-364, LD-376
Integration testing	Validate interaction among frontend, application services, AI service, GitHub, and data storage.	LD-388
Acceptance testing	Validate end-to-end user workflows against approved requirements.	LD-388

Deployment validation	Verify the configured container/deployment workflow.	LD-382
Workflow regression	Recheck important existing behavior after changes.	To be included in the approved test plan; not separately specified in the reviewed issue titles.

2.2 Test Levels

The Jira backlog identifies automated tests and system integration/acceptance testing. Unit, component, integration, system, and acceptance levels should be mapped to actual test suites by the implementation team. Specific coverage targets are not specified in the reviewed Jira items.

2.3 Supporting Tools

Tool / area	Evidence
GitHub Actions	Explicitly tracked by LD-376
CI/CD pipeline	Tracked by LD-364 and LD-358
Docker/deployment	Tracked by LD-382
Automated test framework	Specific framework is not stated in the reviewed Jira summaries
API test tooling	Not specified in the reviewed Jira summaries
Load/security testing tools	Not specified in the reviewed Jira summaries

3. Test Plan

3.1 Human Resources

Individual tester assignments are intentionally omitted. The project should identify the required test roles without publishing personal assignment or contribution details in this report.

3.2 Test Environment

The approved test environment, deployment URLs, database setup, test accounts, GitHub test repository, AI configuration, and secrets-management process are not specified in the reviewed Jira issue summaries.

3.3 Test Milestones

Jira includes the following test and delivery work items:

Milestone / work item	Jira reference
CI/CD pipeline	LD-364
Automated testing	LD-370
GitHub Actions integration	LD-376
Docker and deployment	LD-382
System integration and acceptance testing	LD-388

Due dates and exit criteria should be taken from the current Jira sprint/issue planning.

4. Test Cases

The Jira summaries reviewed for this report do not provide a complete approved test-case catalogue. The following are **candidate scenarios derived from requirements** and must be refined into executable tests.

Test ID	Scenario	Expected result	Traceability
TC-001	Submit valid registration information	Account is created or a successful result is returned.	LD-15, LD-20
TC-002	Attempt authentication with invalid credentials	Access is denied and a suitable error is reported.	LD-16, LD-38
TC-003	Connect a GitHub account	Authorization result is handled and connection state is reported.	LD-17, LD-22, LD-43, LD-53

TC-004	Synchronize an accessible repository	Synchronization result is reported.	LD-24, LD-63
TC-005	Request documentation generation	A result or job status is made available.	LD-80, LD-90, LD-340
TC-006	Request status for an unknown generation job	The system reports that the job cannot be found or resolved.	LD-106, LD-109
TC-007	Process a source change	Potentially affected documentation is identified where supported.	LD-112, LD-113, LD-116, LD-122
TC-008	Analyze possible documentation drift	Findings and available classification/severity information are returned.	LD-140–LD-156
TC-009	Compare documentation versions	The system displays available differences.	LD-174, LD-175, LD-183
TC-010	Submit documentation for review	The submission becomes available to the review workflow.	LD-229, LD-233
TC-011	Approve or reject documentation	The authorized decision is recorded.	LD-230, LD-239

TC-012	Run CI checks	Pipeline results indicate which configured checks pass or fail.	LD-364, LD-370, LD-376
TC-013	Run system integration and acceptance tests	Results are recorded against approved end-to-end criteria.	LD-388

5. Test Reports

A completed test report should record the build/version tested, environment, test case, expected result, actual result, pass/fail status, defect reference, and execution date. A final test report and signed acceptance evidence are not established by the reviewed Jira summaries.

VI. Release Package and User Guides

1. Deliverable Package

The deliverable package should be reconciled with the implementation and the current Jira statuses. The tracked delivery areas include:

- Frontend interfaces (LD-286–LD-316).
- AI service (LD-322–LD-352).
- CI/CD and automated testing (LD-358–LD-376).
- Docker/deployment (LD-382).
- System integration and acceptance testing (LD-388).
- Documentation requirements and workflows (LD-7–LD-13).

This list represents the intended workstreams, not confirmation that every item is included in a release package.

2. Installation Guides

2.1 System Requirements

The final minimum CPU, RAM, disk, operating-system, runtime, database, and network requirements are not specified in the reviewed Jira issue summaries. Confirm these against the actual Docker configuration, dependency files, and deployment environment before publishing installation instructions.

2.2 Installation Instructions

The Jira backlog tracks Docker and deployment under LD-382 but does not provide a verified step-by-step installation guide. The final guide should be generated from the actual repository and deployment configuration and include:

1. Prerequisites and supported runtime versions.
2. Repository checkout instructions.
3. Environment-variable and secret configuration without publishing secret values.
4. Container/build instructions.
5. Database setup and migration steps, if applicable.
6. Service startup and health-check verification.
7. Automated test commands and expected results.
8. Troubleshooting and shutdown instructions.

Commands and configuration values must be copied from the current repository rather than invented in this report.

3. User Manual

The following workflow outlines are based on the Jira use cases. Screenshots and exact button labels should be added after validating the actual UI.

3.1 Developer / Project Member Workflow

1. Register or sign in.
2. Connect a GitHub account and select an accessible repository.
3. Synchronize repository information where supported.
4. Request documentation generation.
5. Preview and edit the generated result.
6. Inspect available evidence or confidence information.
7. Submit documentation for review if required.
8. Search documentation, browse code/documentation relationships, or view version history.
9. Review pipeline results when relevant.

Related Jira areas: LD-15–LD-106, LD-201–LD-223, LD-364–LD-376.

3.2 Reviewer / Approver Workflow

1. Open documentation submitted for review.
2. Review the content against the agreed requirements.
3. Add comments or request changes when needed.
4. An authorized approver approves or rejects the submission.
5. Consult the approval audit trail where permitted.

Related Jira areas: LD-229–LD-251.

3.3 Administrator Workflow

1. Manage users and role assignments.
2. Manage workspaces, repositories, and access permissions.
3. Configure supported system and integration settings.
4. Monitor jobs, audit events, and notifications.
5. Inspect CI/test results and deployment status where available.

Related Jira areas: LD-257–LD-280, LD-364–LD-388.

3.4 AI Documentation Maintenance Workflow

1. Detect source changes.
2. Identify documentation potentially affected by the changes.
3. Analyze possible documentation drift.
4. Review classifications, severity, and available AI suggestions.
5. Generate or accept a documentation update through the supported workflow.
6. Inspect the change log or compare versions.
7. Roll back to an eligible previous version if the approved workflow allows it.

Related Jira areas: LD-112–LD-195, LD-322–LD-352.

Appendix A. Jira Traceability

A.1 Epic-Level Traceability

Epic	Area	Purpose
LD-5	Authentication & User Management	User authentication and access-related functions
LD-6	GitHub Integration	Connect and synchronize repositories and source-change information
LD-7	Documentation Generation	Analyze source code and generate documentation
LD-8	Automatic Documentation Update	Detect source changes and maintain affected documentation

LD-9	Documentation Drift Detection & Review	Identify and resolve source/documentation mismatches
LD-10	Documentation Version & Change Log	Track versions, changes, comparisons, and rollback
LD-11	Knowledge Base	Search and navigate documentation-related knowledge
LD-12	Review & Approval Workflow	Review, comment, approve/reject, and audit decisions
LD-13	Administration & Governance	Manage users, roles, workspaces, configuration, and monitoring
LD-286	Frontend Implementation	Build user interfaces for core functions
LD-322	AI Service Implementation	Code analysis, retrieval, generation, drift analysis, and AI suggestions
LD-358	CI/CD & System Testing	Pipeline, automated tests, deployment, and system acceptance

A.2 Status Interpretation

Jira statuses are dynamic and should be checked directly before a project review or release. In the reviewed snapshot, some implementation tasks were marked Done, some were In Review, and many remained To Do. This document deliberately avoids treating an epic or planned feature as complete solely because its name appears in the backlog.

A.3 Source and Maintenance Note

This report is a Jira-based consolidation aligned to the major chapter structure of the supplied PDCMS report. Before submission, the project team should validate the requirement wording, diagram relationships, actual architecture, data model, non-functional targets, test

evidence, and installation instructions against the implementation and any approved design documents.